

## Notes Week 27

### a. Correspondence with Amanda Karakas

“

Hi Amanda,

Thanks for your reply and sharing the folder with all of the pulse files!

After looking through the files and discussing with Onno, we do however have one more question that we hope you can help us with. I will briefly describe how we arrived at the question below.

I took some time to match the pulses in the pulse.dat files with the pulses in the intershell abundances files from Zara for all models. This allows me to associate the dredged-up mass from each pulse with the corresponding intershell abundances. To check that this matching is correct, I plotted a quantity that is present in both datasets for every thermal pulse: the core mass.

”

The top plot shows the core mass  $M_{\text{core}} (M_{\odot})$  on the y-axis (ranging from 0.54 to 0.68) versus the thermal pulse count on the x-axis (ranging from 0 to 32). It displays several curves for different models: M = 3.0 (blue), M = 2.75 (orange), M = 2.5 (green), M = 2.25 (red), M = 2.0 (purple), and M = 1.75 (brown). Thick transparent lines represent core mass from intershell files, and thin opaque lines represent core mass from pulse.dat files. The bottom plot shows the residual  $M_{\odot}$  on a logarithmic y-axis (from  $10^{-7}$  to  $10^{-5}$ ) versus the thermal pulse count. It shows the difference between the two datasets, with most residuals falling between  $10^{-6}$  and  $10^{-5}$ .

In the top plot, the thick transparent lines correspond to the core mass per thermal pulse as listed in the intershell files, while the thin opaque lines show the core masses from the pulse.dat files. I think the close agreement between the two shows that the pulses in the two datasets correspond to the same thermal pulses.

This suggests I should be able to combine the dredged-up mass from the pulse.dat files with the corresponding intershell abundances from the intershell files and compare them.

However, when I plot the dredged-up mass per thermal pulse together with the total mass fraction of elements heavier than iron in the intershell, I see something that Onno and I did not expect:

The top plot shows the sum of mass fractions  $\sum_{Z>26} X_i$  on the y-axis (scaled by  $10^{-4}$ , ranging from 1.0 to 4.0) versus the thermal pulse count on the x-axis (ranging from 8 to 32). It shows curves for the same models as the first plot. The bottom plot shows the dredged-up mass  $M_{\text{DUP}} (M_{\odot})$  on a logarithmic y-axis (from  $10^{-5}$  to  $10^{-2}$ ) versus the thermal pulse count. Both plots show that s-process enrichment (indicated by the top plot) and dredged-up mass (indicated by the bottom plot) occur at different times for different models, with some models showing enrichment after the first TDU and others after subsequent TDUs.

Onno and I expected the intershell to become enriched in s-process elements 1 pulse after the first thermal pulse with significant amounts of dredge up (like what we see for the brown, green, and red models). However, in some models the intershell only becomes enriched after a second TDU (blue and purple models) or even after a third or fourth TDU (pink and orange models). We see similar offsets in the models with different metallicities as well.

Do you happen to know what might cause some models to only experience s-process enrichment in the intershell after multiple TDU events?

Thanks again for your help!

Best regards,

Koen

“

Hi Koen,

I took some time to match the pulses in the pulse.dat files with the pulses in the intershell abundances files from Zara for all models. This allows me to associate the dredged-up mass from each pulse with the corresponding intershell abundances. To check that this matching is correct, I plotted a quantity that is present in both datasets for every thermal pulse: the core mass.

OK one thing to be aware of is that the time when the core mass was sampled may be different in the pulse.dat files and the intershell abundance files. In the pulse.dat files the core mass is sampled at the luminosity maximum, which is during the interpulse but just before the next thermal pulse. In the intershell files I think the core mass is sampled during the thermal pulse itself, so there may be a slight offset.



In the top plot, the thick transparent lines correspond to the core mass per thermal pulse as listed in the intershell files, while the thin opaque lines show the core masses from the pulse.dat files. I think the close agreement between the two shows that the pulses in the two datasets correspond to the same thermal pulses.

This suggests I should be able to combine the dredged-up mass from the pulse.dat files with the corresponding intershell abundances from the intershell files and compare them.

Howevew, when I plot the dredged-up mass per thermal pulse together with the total mass fraction of elements heavier than iron in the intershell, I see something that Onno and I did not expect:



Onno and I expected the intershell to become enriched in s-process elements 1 pulse after the first thermal pulse with significant amounts of dredge up (like what we see for the brown, green, and red models). However, in some models the intershell only becomes enriched after a second TDU (blue and purple models) or even after a third or fourth TDU (pink and orange models). We see similar offsets in the models with different metallicities as well.

Do you happen to know what might cause some models to only experience s-process enrichment in the intershell after multiple TDU events?

I will investigate this for you but one thing does come to mind:

We only get s-process production after we manually include some protons into the top of the He-intershell. This partially mixed zone of protons turns into a c13 pocket. We generally insert protons at the deepest extent of a TDU episode, with the first insertion occurring after the first thermal pulse with TDU. But in some cases I might ignore the first TDU episode if it's not very deep, or, perhaps the algorithm that finds the TDU episodes might miss the first one. That can also happen.

I will examine the m1.75, m2.75 and m2.5 models (brown, orange and green in your plots above) to verify what is going on.

Cheers,

Amanda

”

“

Hi again,

I meant I will examine the models where the s-process appears after the 2nd or 3rd TDU events! sorry mis-read your email. I'm checking now.

cheers,

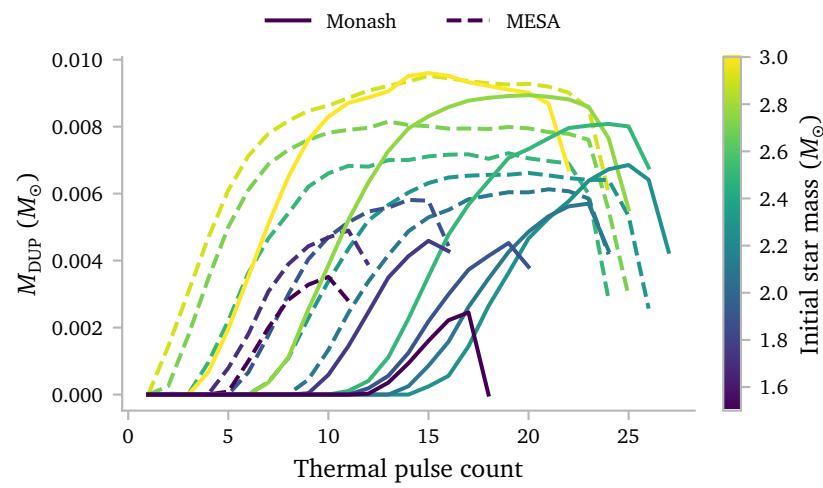
Amanda

”

## b. Comparing MESA and Monash Dredge up

My first idea estimate the intershell abundances of the MESA models by parametrising the intershell abundance as a function of cumulative dredge up. This makes little physical sense, but is convenient because the cumulative amount of dredge up is strictly increasing.

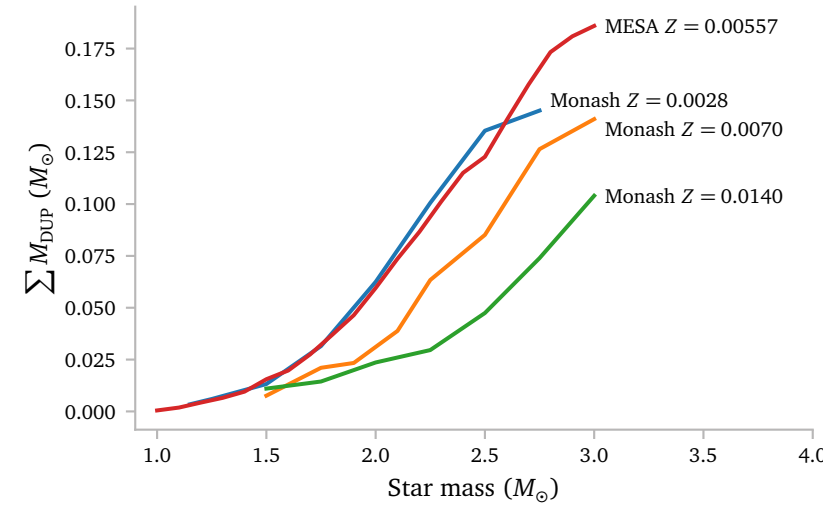
However, this is only a good way to approximate the intershell abundances if the dredge up episodes of the MESA models are in good agreement with the dredge up episodes of the Monash models.



It is important to note that I plot the Monash models with  $Z = 0.007$  and MESA models with  $Z = 0.00557$ , so there is a slight difference there.

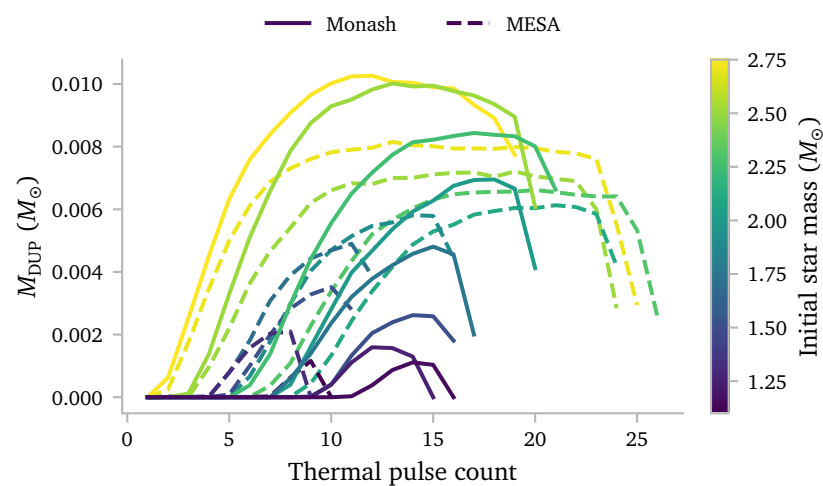
It seems like the MESA models generally undergo dredge up earlier. The magnitudes are quite similar, but because the MESA models start dredge up earlier, I think they will in general experience a bit more total dredge up

Below, I plot the total amount of dredge up for the various Monash and MESA models.



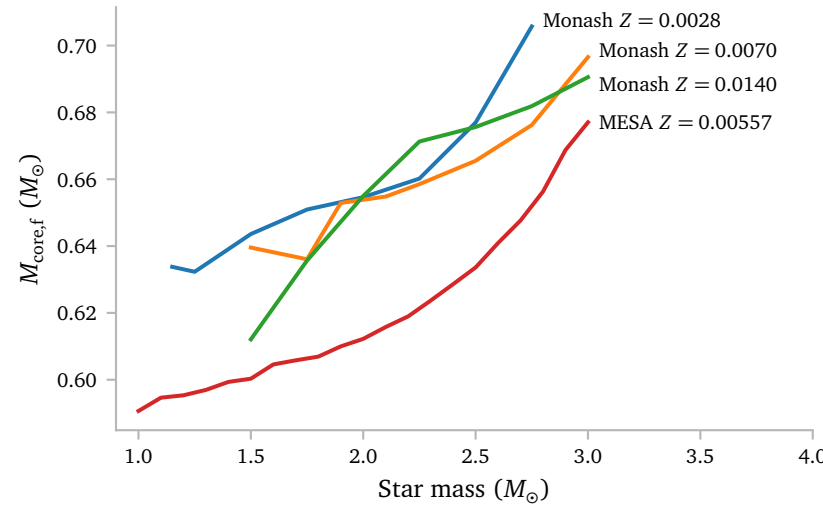
The MESA models agree very well with the lower  $Z = 0.0028$  metallicity Monash models. Obviously, since these are entirely different codes, it is to be expected that they do not predict the same amount of dredge up, but I feel like the difference here and in the previous plot is actually quite significant, especially if I use the cumulative dredged up mass to predict the intershell abundance.

Since the Monash  $Z = 0.0028$  models seem to agree very well with the MESA models when it comes to total dredged up mass, I want to compare them pulse by pulse as well.



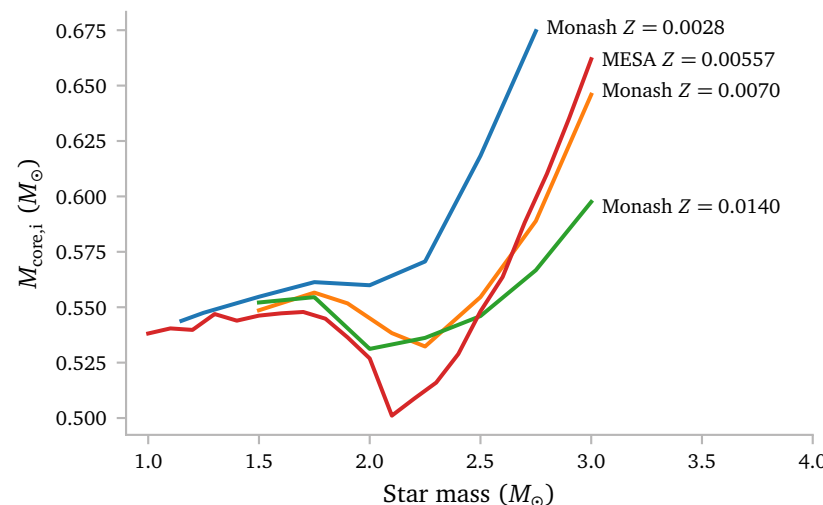
It seems like it is a coincidence that the cumulative amount of dredged up mass agrees so well. The MESA models generally experience more thermal pulses with dredge up, while the amount of dredged up mass per pulse is lower.

Apart from the amount of dredged up mass, there are some other properties that we can compare between the models. First, let me show the final core mass as a function of stellar mass:

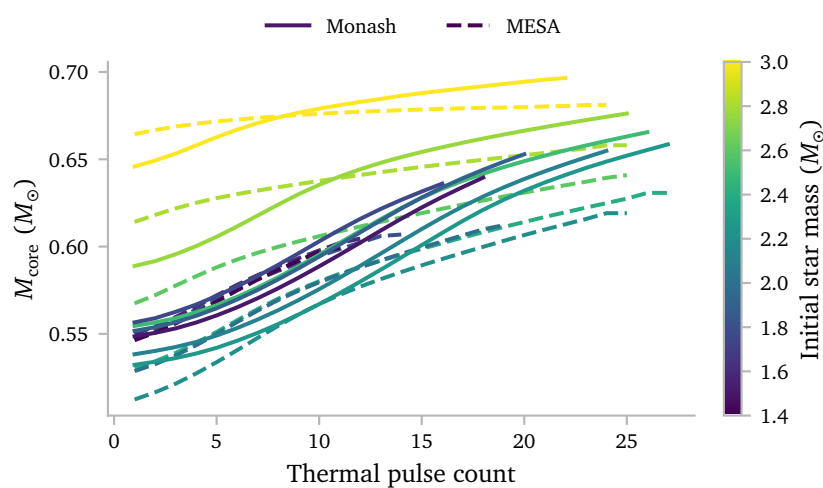


The final core mass of the MESA models is always lower than that of the Monash models. It think it is not super surprising given the fact that we saw that the MESA models experience *more* dredge up, and thus grow their core *slower*.

I think it is interesting to look at the initial core mass too though, because this is a good check if the pre-TPAGB evolution could be a cause for this difference too.



The initial core mass is also not identical, but at least for the higher masses, MESA and Monash agree well. I do think it is interesting that the core mass of the MESA models is much lower for the models that just miss the core Helium flash.

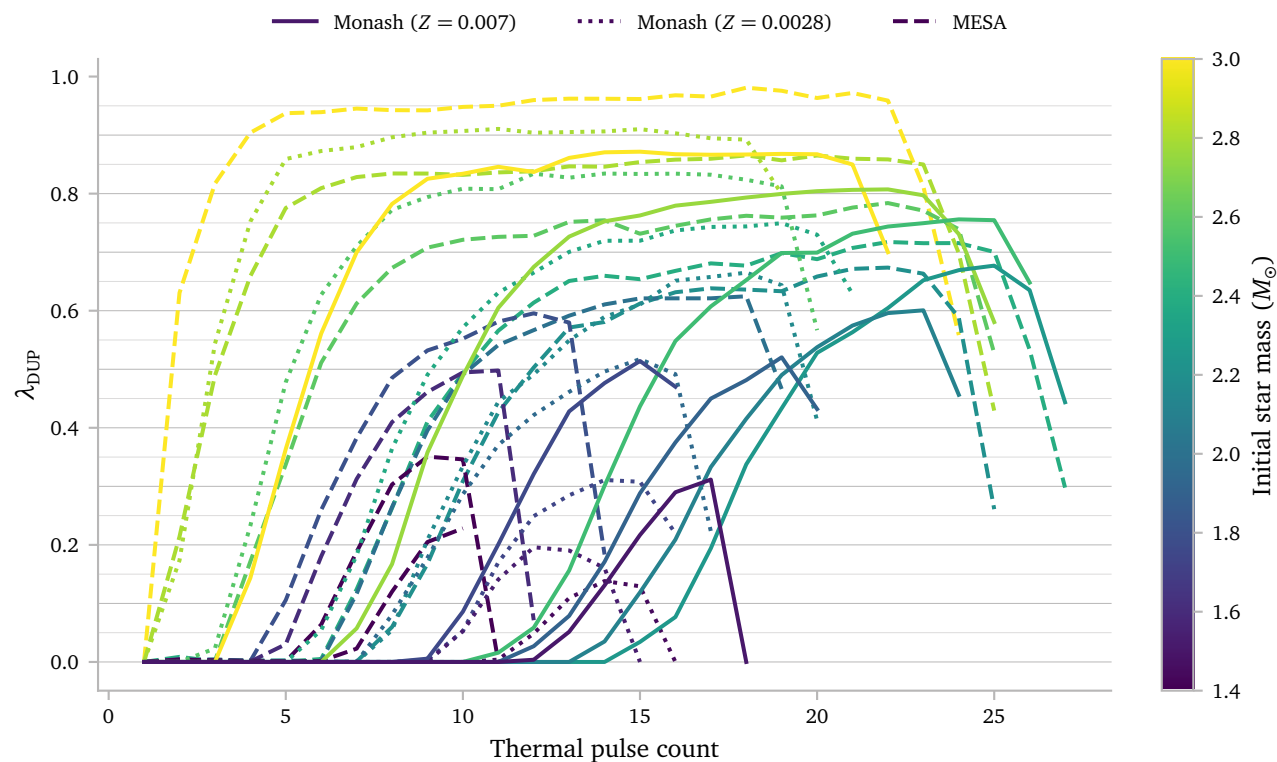


The core mass evolution of the MESA models is *much* more flat than that of the Monash models. This is exactly what we expect based on the lower total dredged up mass.

I do think it is a bit surprising that we find such large differences especially in the core mass, since Cinquegrana et al. (2022) found only very small differences in the core mass of a  $2 M_\odot$   $Z = 0.03$  star. However, they did specifically try to match every parameter of Monash to the equivalent MESA parameter, which we did not do.

Rees et al. (2024) also looked at the core masses of MESA models and Monash models. Just like us, they also find that for low mass TPAGB stars, the MESA core masses are lower than those of the Monash models. They do note that MESA and Monash have slightly different definitions of the core mass, however, in both cases it is based on the hydrogen burning shell. They note that, since the hydrogen burning shell is very thin in AGB stars, the differences in the core masses are *not* due to different definitions.

The plot below shows the dredge up efficiency parameter  $\lambda$  for the MESA models, as well as for the  $Z = 0.0028$  and  $Z = 0.007$  Monash models.

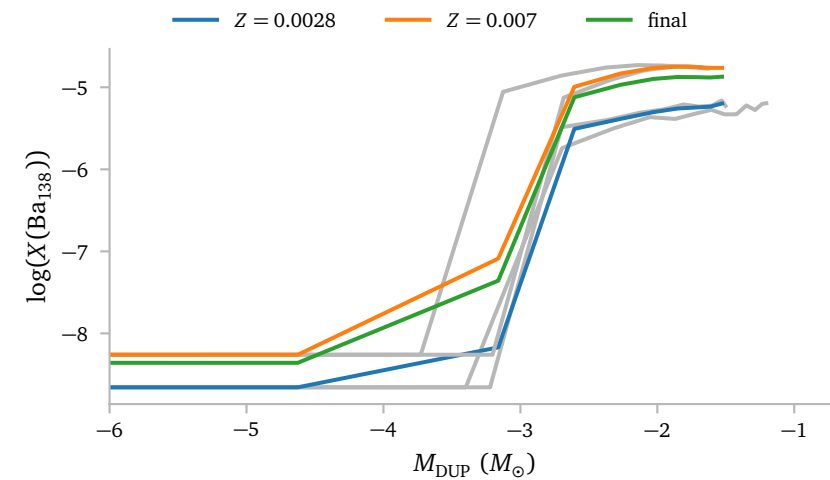


The shapes here are actually quite similar for all models. It is just that the MESA models seem to predict higher overall values and obviously start during earlier pulses.

## c. Comparing different Intershell abundance parametrisations

### c.1 Summary of Different Methods

1. *Dredged-up mass*: This method parametrises the isotope intershell abundance only based on the total amount of dredged-up mass. This is nice because, just like for example the thermal pulse count, this quantity is strictly increasing, and it has some physical meaning since we expect the intershell abundance enrichment to only start increasing after a dredge-up has started. However, it makes little physical sense to say that the intershell abundance is dependent on the *cumulative* amount of dredged up mass. Really, the only important thing is the *first* time significant dredge up occurs.



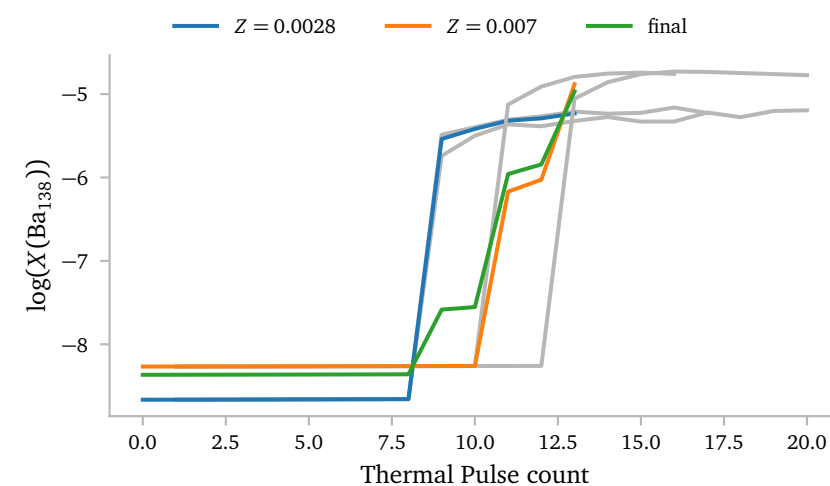
We see that this method is quite good at interpolating the latter part of the isotope abundance evolution as a function of cumulative dredged-up mass. However, there is not a very well defined “start” of the intershell abundances *because* the abundances increase very quickly after the first or second dredge-up event. These events can dredge up quite different amounts of mass.

2. *Naive thermal pulse*: This method only looks at the intershell abundance of an isotope as a function of the thermal pulse number. We then *assume* that the MESA and Monash models agree well on when the intershell abundances change as a function of thermal pulse number.

This is probably *not* the case, but it is an interesting simple test case. I assume that this method is not very accurate because the MESA and Monash models do not agree on the total number of thermal pulses, and they do not agree on when the first pulse with significant dredge up occurs.

Another reason why I think this method is not ideal is that we *need* to interpolate intershell abundances of the different monash models, and even these models do not agree well on during which thermal pulse enrichment occurs. This means that interpolation will give “smoothed-out” results.

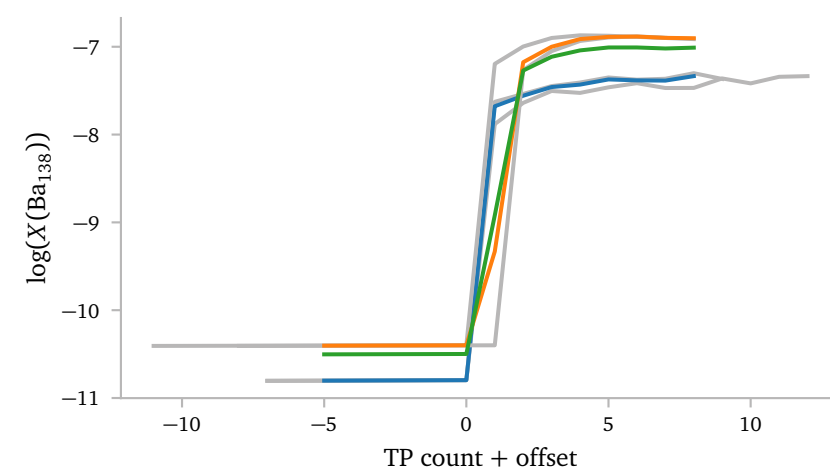
Below I show how a  $M = 1.8 M_{\odot}, R = 500 R_{\odot}, q = 0.4293$  models gets interpolated:



Here we see the “problem” with this method. Since for the same metallicity, the different masses do not agree on the moment of enrichment, the first interpolation creates a smoothed intershell abundance enrichment profile for the  $Z = 0.007$  models. Then when we interpolate the different interpolated  $Z$  models, we get an even more smoothed out profile. This is *not* representative of how the single model intershell enrichment looks.

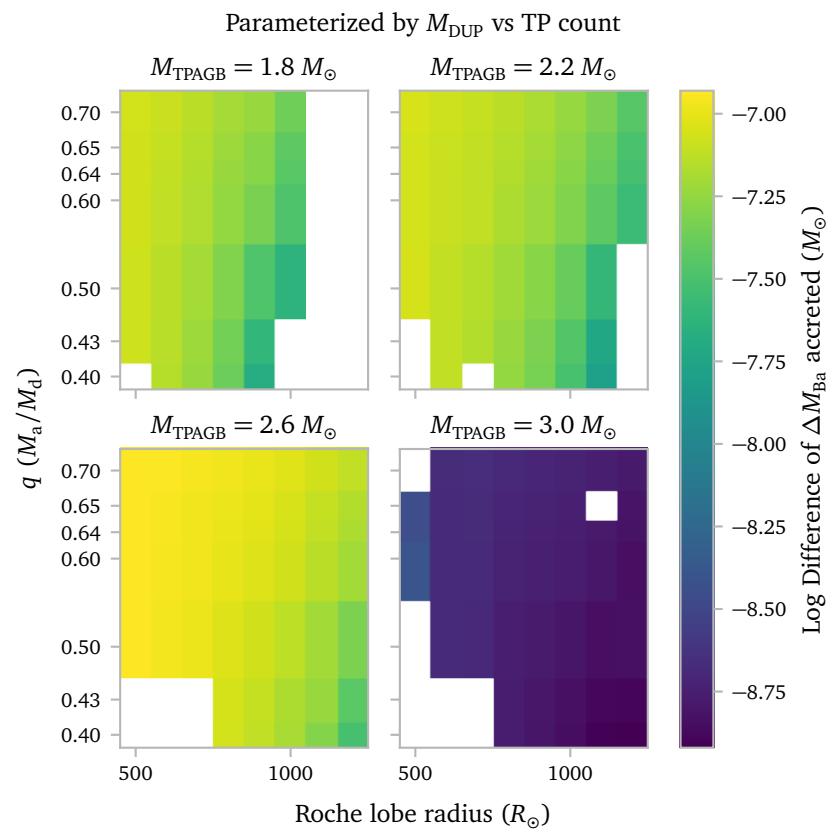
3. *Thermal pulse shifted by first significant dredge-up*: Both of the previously noted methods clearly have problems. We can try to take the best parts of both. Let’s take the simplicity of the thermal pulse count idea and the physical meaning of the first dredge-up that is so powerful in the first method.

For now I put a simple lower limit of  $\log M_{\text{DUP}} = -4.5 M_{\odot}$  for the start of dredge up. I then offset the thermal pulse count by the first pulse that satisfies the lower boundary.



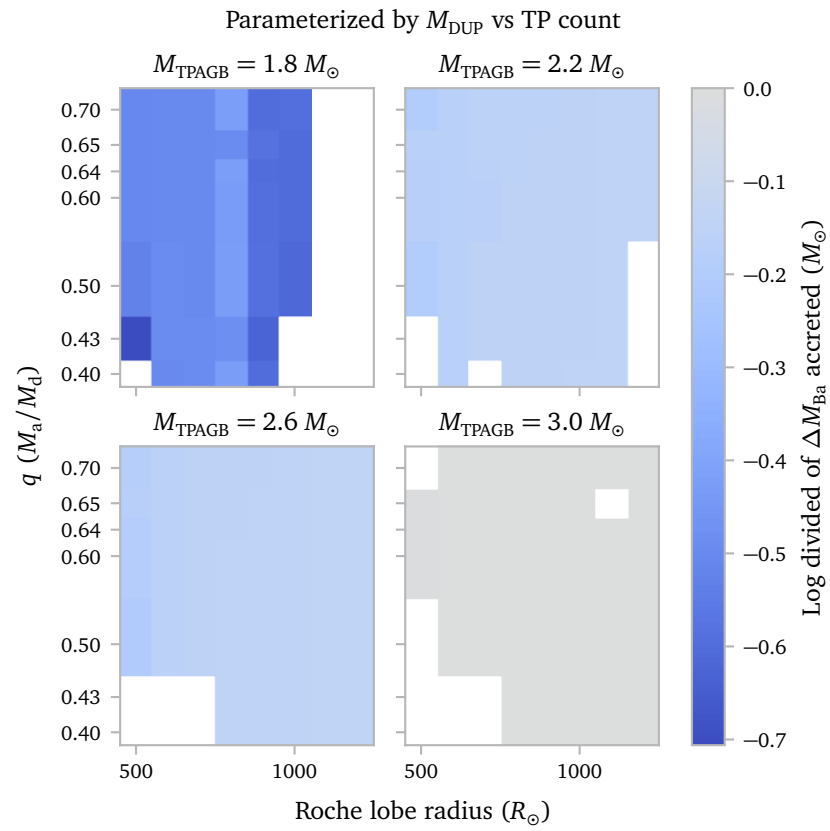
I think this looks quite good. The models however do not perfectly agree on when the intershell should become enriched. We have already seen this before and I have asked Amanda about this, but at this point I have not gotten a response yet.

### c.2 Comparing Naive Thermal Pulse parametrisation with Cumulative Dredged-up Mass parametrisation



It is clear that the absolute differences between the two methods for determining the intershell abundance, and thus the total amount of accreted barium is smallest for the  $M_{\text{TPAGB},i} = 3 M_{\odot}$  model and largest for the two lightest models. However, we do not know what the absolute values of  $\Delta M_{\text{Ba}}$  are. Therefore, I also plot the log-divided difference between the methods.

Here, I plot  $\log(M_{\text{dup method}} - \text{TP count method})$

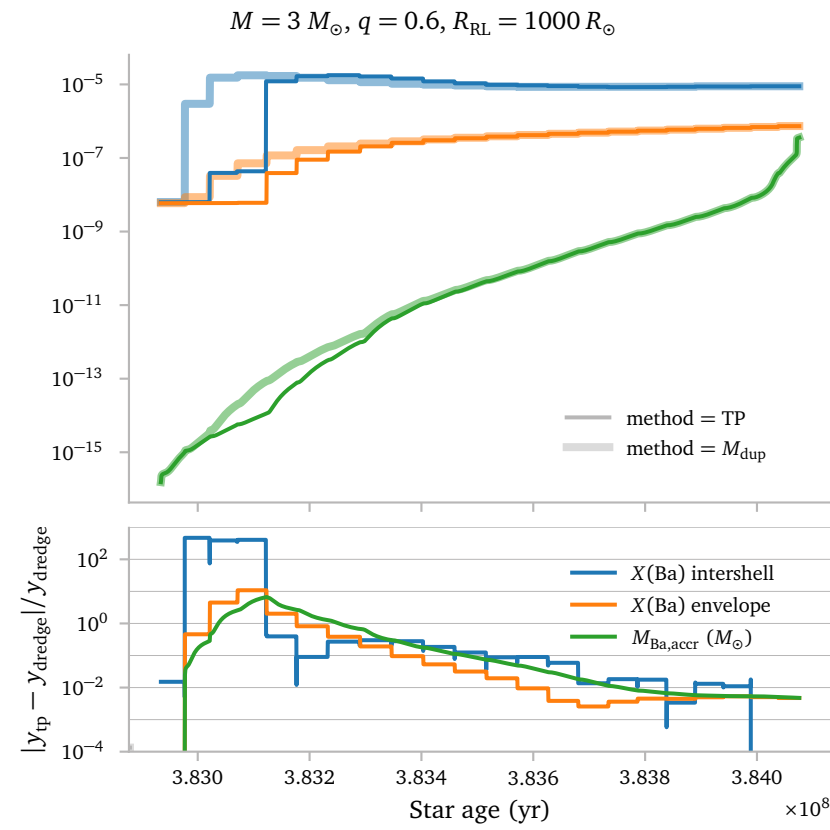


This plot shows  $\log(M_{\text{dup method}}/\text{TP count method})$

This time it is more obvious that the differences are tiny for the  $M = 3 M_{\odot}$  models and largest for the  $M = 1.8 M_{\odot}$  models.

Now the question is, *why* do these differences exist?

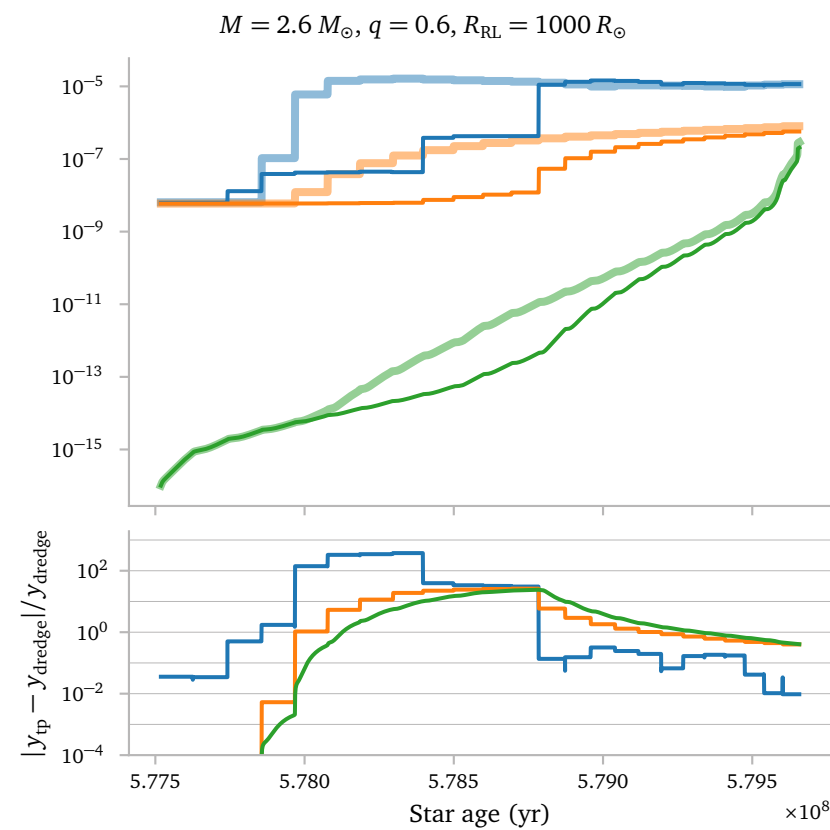
First, let me show a  $3 M_{\odot}$  model.



Now as we could see in the previous figure, the  $M = 3 M_{\odot}$  models have very little differences between the amount of accreted barium mass.

I think this is because in these models, dredge up starts early in both the MESA and the Monash models, such that overall, the differences in the *envelope* abundances remain small after some dredge up events. This also means that as soon as the star starts to lose significant amounts of mass, well after dredge up has begun, the envelope abundances agree well and we are left with roughly the same amount of accreted barium.

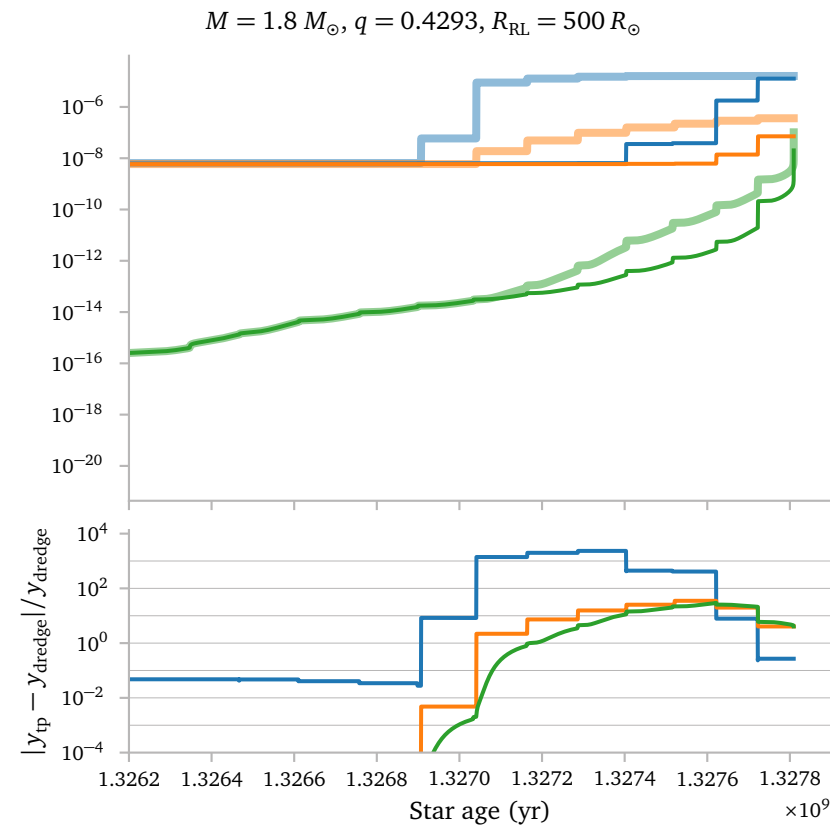
Now this is not only true for barium, but pretty much all s-process elements, because the enrichment happens in the same way.



In the differences plot above, we see that the  $M = 2.6 M_{\odot}$  models generally have lower values by about 1.5 to 2.

Here, we see that it takes a lot longer for the intershell to become enriched. It also seems like it takes many pulses for the intershell to actually reach its final intershell abundance maximum, while the Monash models reach this maximum in only 1 or 2 pulses. This is a result of the interpolation which is definitely not perfect.

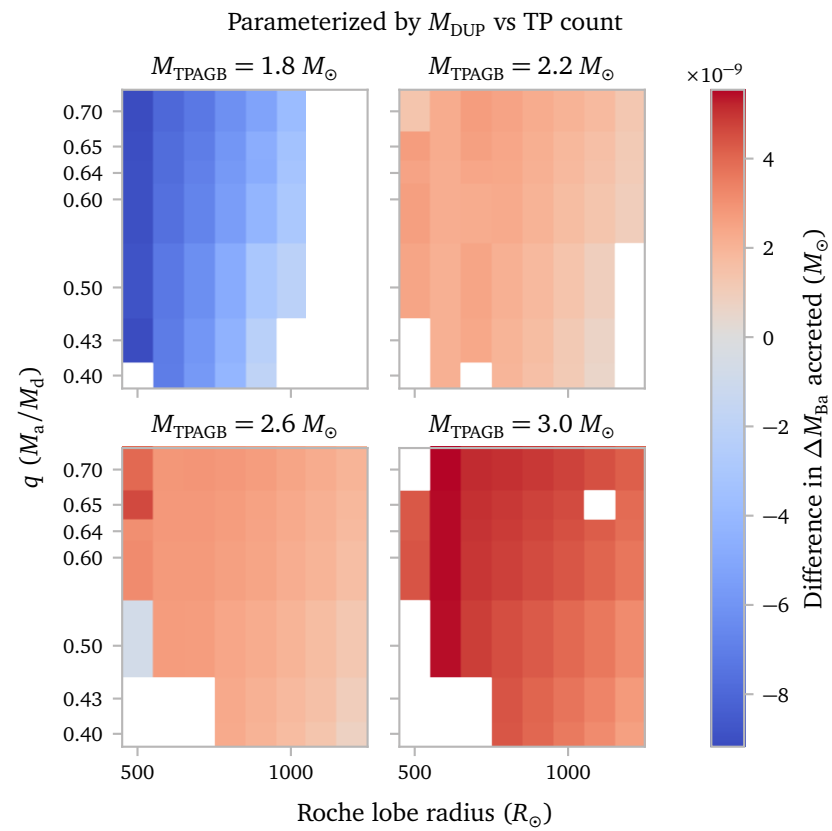
Luckily, there are *many* pulses after the first TDU event. This way, even though the envelope is initially much less enriched, it can “catch” up before significant amounts of mass are transferred. Once the strong winds and RLOF start, the envelope is nearly at the same enrichment as it is for the  $M_{\text{DUP}}$  interpolation method.



This model has the largest fractional difference between the methods. It is quite clear that for this  $1.8 M_{\odot}$  model, the difference between when the intershell becomes enriched is quite large. This means the intershell becomes enriched much earlier in the  $M_{\text{DUP}}$  method than it does in the TP count method. However, because this model loses most of its mass very quickly after the intershell enrichment of the TP count method, it does not have any time to properly enrich its envelope, while the  $M_{\text{DUP}}$  method did have time to enrich it.

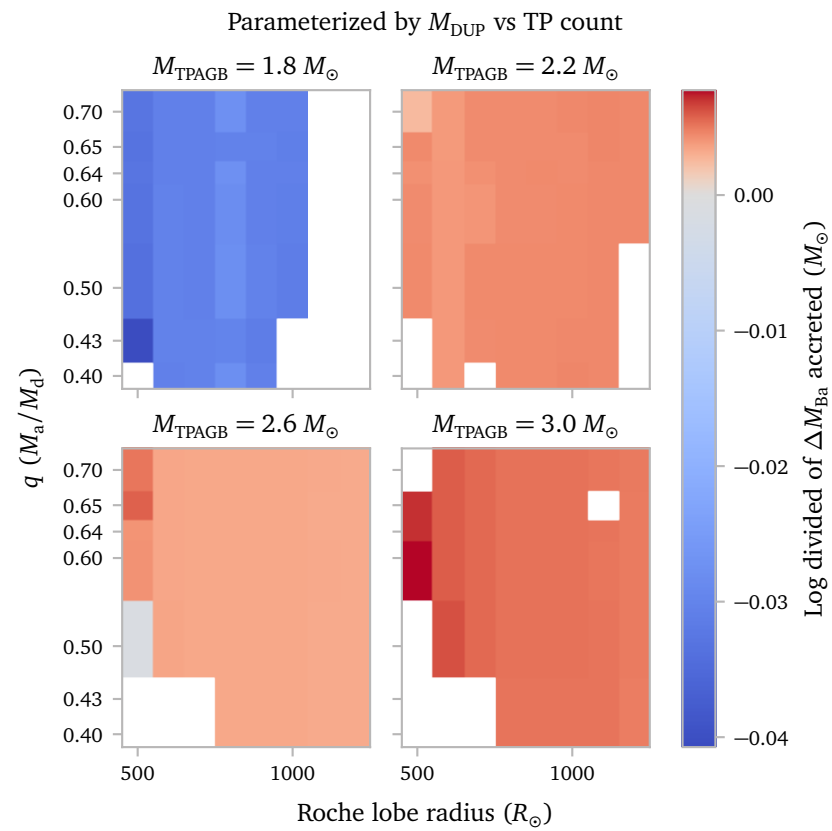
### c.3 Comparing Thermal Pulse Offset parametrisation with Cumulative Dredged-up Mass parametrisation

The first subsection already showed that the offset thermal pulse method produces enrichment profiles that are quite similar to the ones from the cumulative  $M_{\text{DUP}}$  method. With the plots below, I show that this is true for *all* models from my test grid.

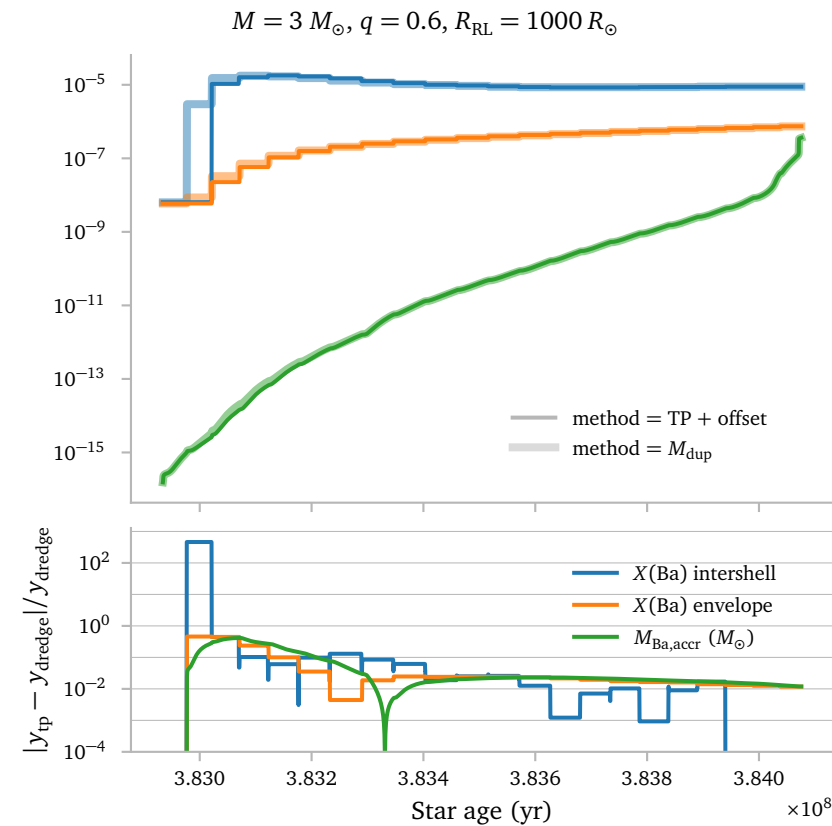


The absolute differences that we see here are *much* smaller than the ones we see in the Naive TP count method. Now, this does not necessarily mean that this method is *better*, it just means it agrees better with the  $M_{\text{DUP}}$  method.

I do like the cumulative dredge-up method better than the naive thermal pulse counting method however for the arguments I have given in the first subsection, so the fact that this offset pulse method agrees much better with the  $M_{\text{DUP}}$  method is quite nice. It means that the precise method for determining the intershell abundance likely is not super important. For agreement, on the first pulse where enrichment occurs seems to be important, and by lining this up with the first TDU event we make sure both methods agree on when this is.

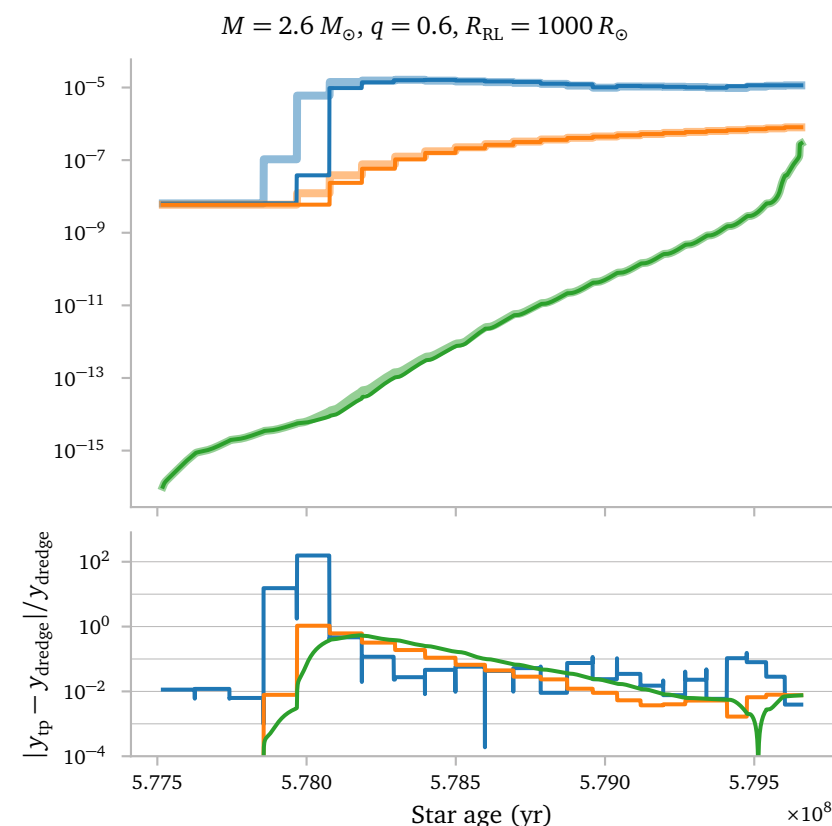


Here we again see much smaller differences between the two methods.



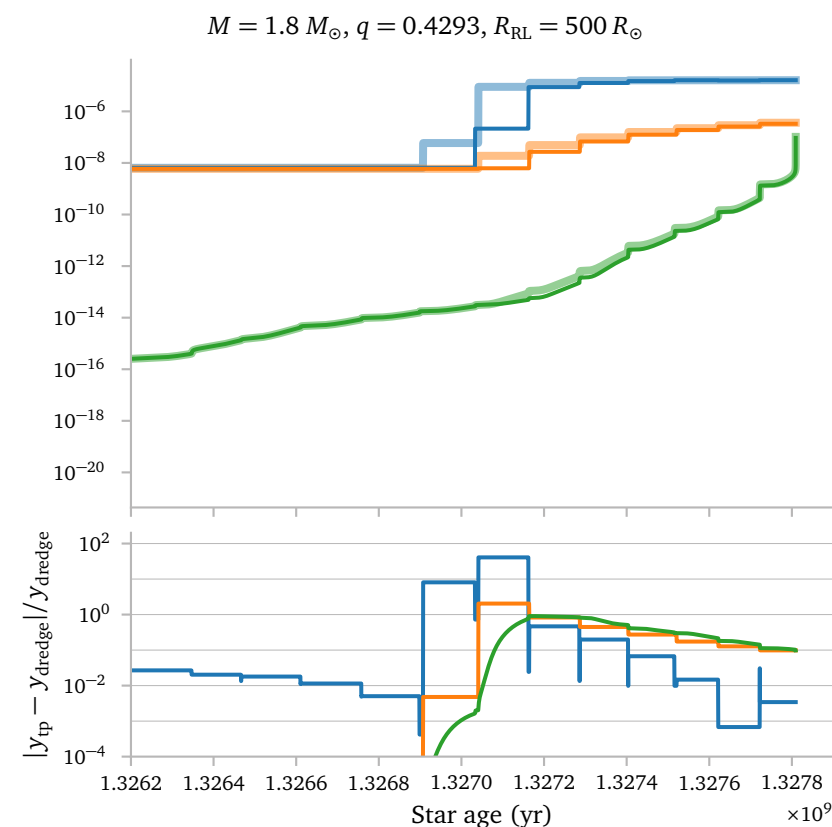
In the  $M = 3.0 M_{\odot}$  models we see that the  $M_{\text{DUP}}$  method stars its intershell (only) one thermal pulse before the TP + offset method. This agreement is quite good, and in stark contrast with the naive TP method where we saw much larger discrepancies.

In the end, the method agrees to within 1 percent on the final amount of accreted barium.



The same is true for the  $M = 2.6 M_{\odot}$  models. Here, again the  $M_{\text{DUP}}$  method enriches the intershell one pulse before the TP + offset method.

Eventually, at the end of the evolution, the differences between the total amount of accreted barium onto the accretor is less than 1%.



Here, I once again plot the model where the two methods disagree *most*. This is a challenging model because dredge up has only *just* started when significant amounts of mass are lost, so this model is most sensitive differences in the starting pulse of intershell enrichment.

Again, the methods disagree on the starting pulse by one. In this case however, this leads to quite a big difference in the total amount of accreted matter, *because* mass loss is already strong at the point of intershell enrichment *and* the model undergoes RLOF really quickly after the enrichment. The offset in the start then causes a *significant* lag in the envelope abundance and thus the amount of accreted mass. At the end of the evolution, the difference between the methods is about 10%, which is *significantly* less than it was for the naive thermal pulse method.

d. Comparing with MESA abundances

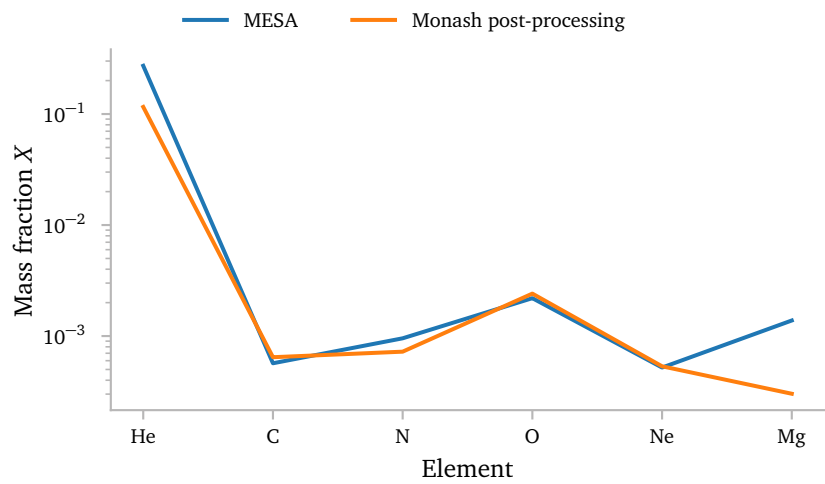
There are some isotopes for which MESA keeps track of the abundances in the envelope. These are:

|                 |                 |                 |                  |                  |
|-----------------|-----------------|-----------------|------------------|------------------|
| H <sub>1</sub>  | He <sub>4</sub> | C <sub>13</sub> | O <sub>16</sub>  | Mg <sub>24</sub> |
| He <sub>3</sub> | C <sub>12</sub> | N <sub>14</sub> | Ne <sub>20</sub> |                  |

These isotopes are interesting, because we can use these to *check* how well the Monash abundance post-processing agrees with the isotopes that MESA keeps track of.

**NOTE:** I think it is important to note that my envelope abundances can only keep track of elements, not individual isotopes. This is because the Monash models only provide elemental abundances in for the envelope, and we *require* an initial envelope abundance which we then enrich based on subsequent dredge-up events.

For example, the first thing we can do is check if the initial envelope abundances agree. For this, I sum He<sub>3</sub> and He<sub>4</sub> and I sum C<sub>12</sub> and C<sub>13</sub>. For the other isotopes, I *assume* these are by far the most abundant isotope for the element, such that  $X(\text{element}) \approx X(\text{isotope})$ .

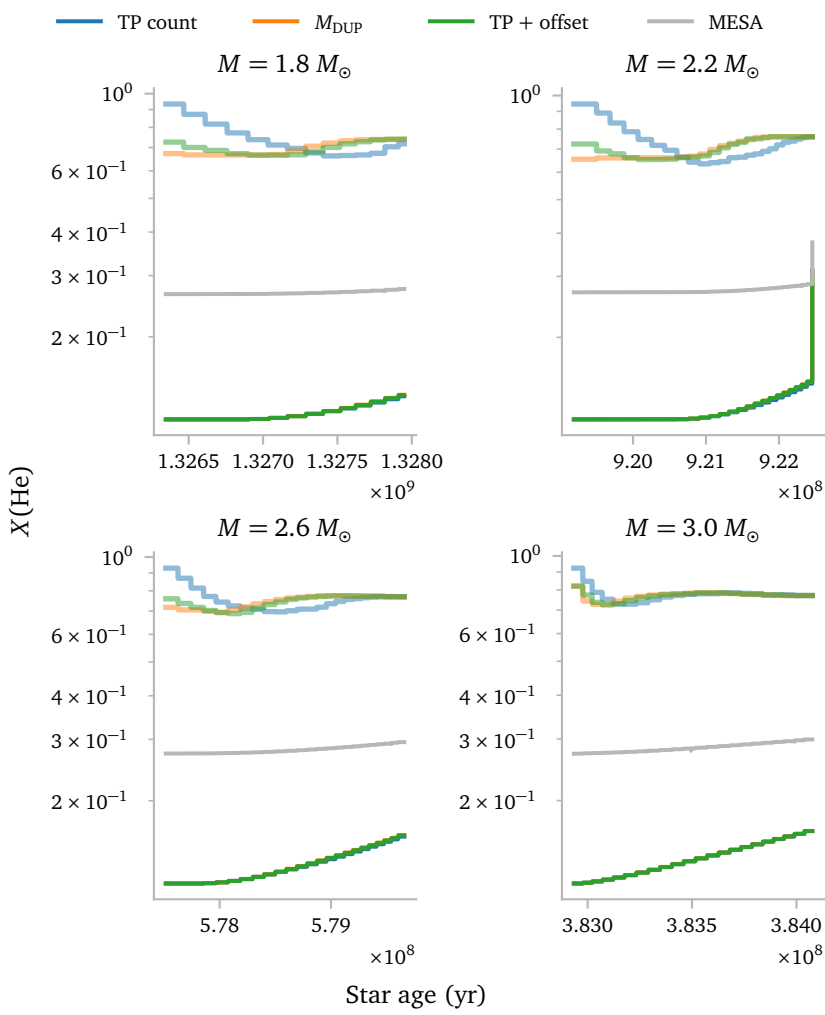


I think the Monash initial envelope abundances agree very well with the MESA initial envelope abundance.

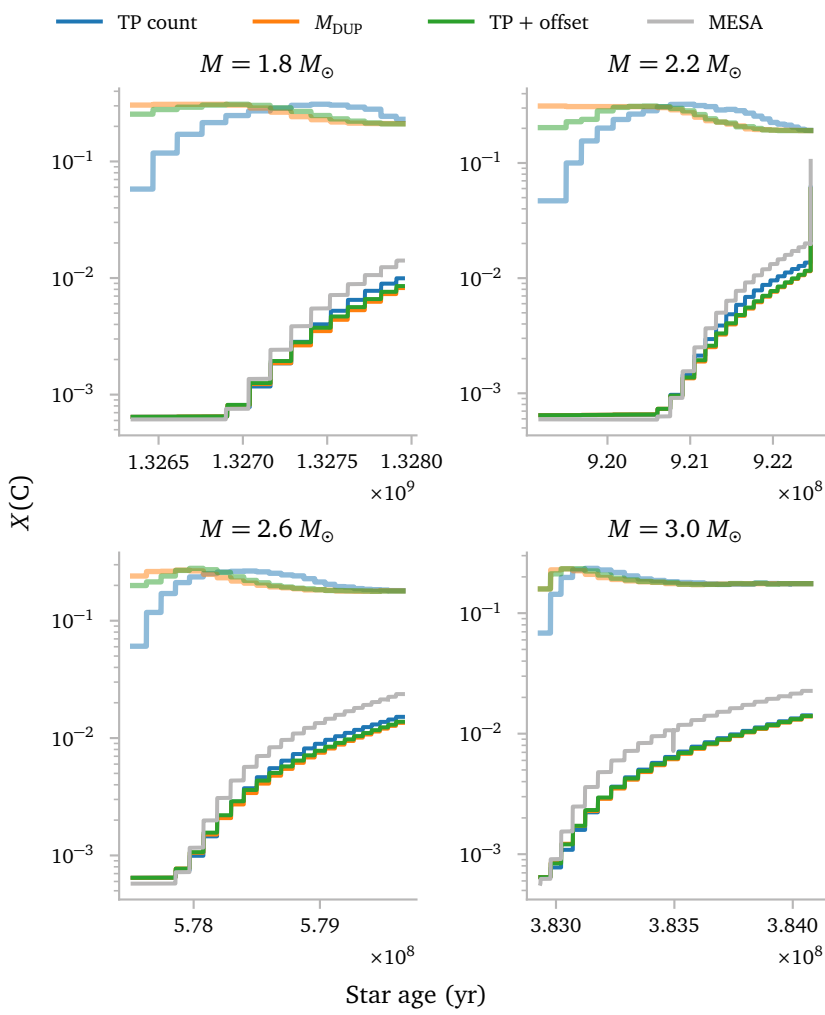
We do however see that the magnesium abundance of the MESA models is quite a bit larger than it is for the Monash abundances. I think this might be because in the monash models, the magnesium atoms are able to undergo interactions and ultimately become heavier elements. In the MESA abundances however, this is not possible because the reaction network only goes up to magnesium.

Now, let me check how the envelope abundances compare between the MESA model and the Monash post-processing.

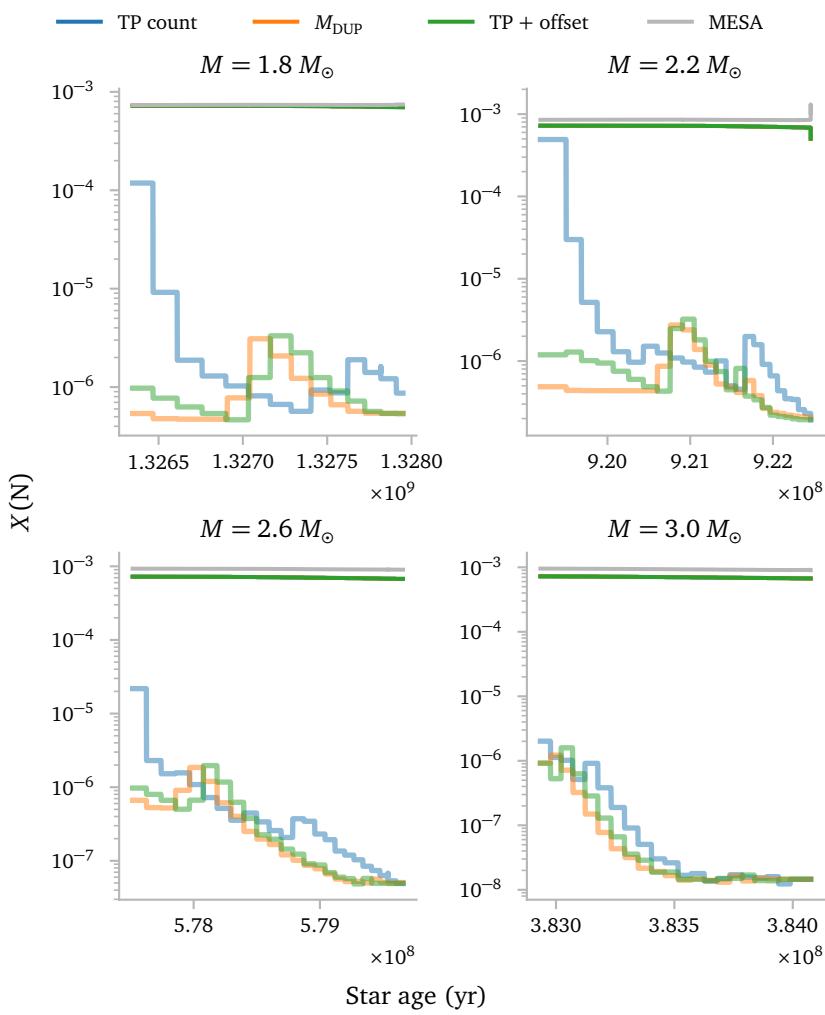
To start, I will look at helium. In this case, all plot are for  $q = 0.6$ ,  $R = 1000 R_{\odot}$ :



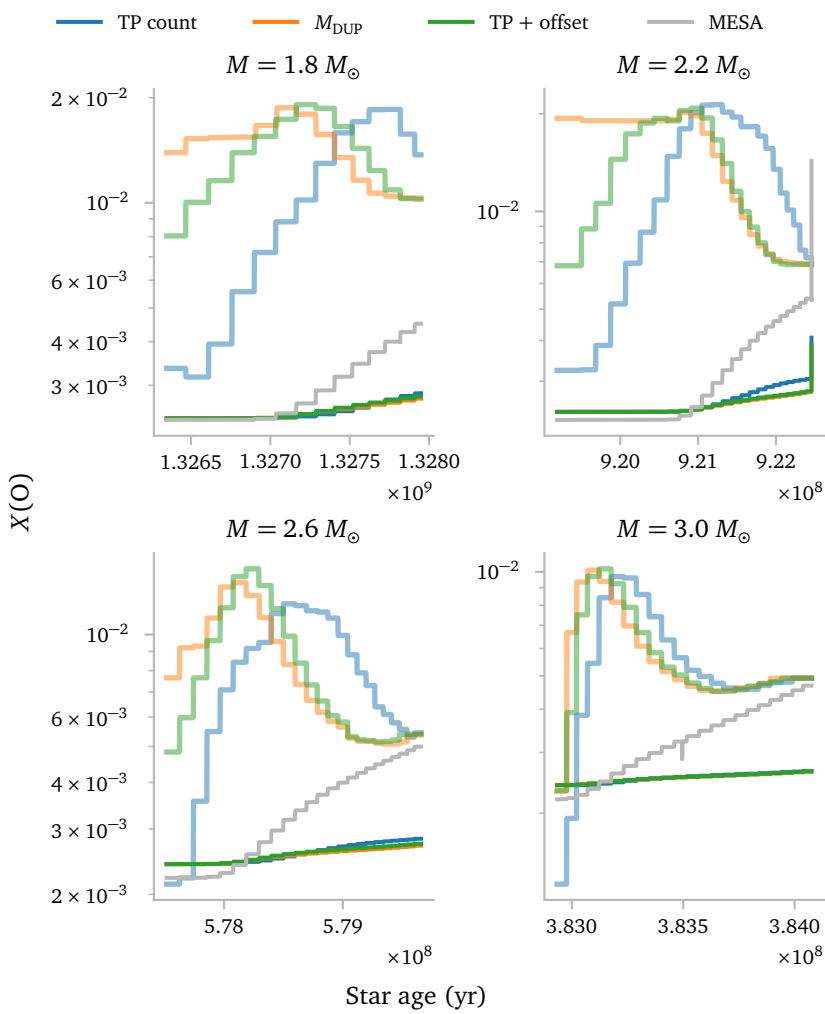
The initial abundance is off by a little more than a factor 2. Since the intershell abundance is larger than both envelope abundances, the envelope abundances grow with every thermal pulse. It is difficult to say if the actual increase in envelope abundance between the two is consistent however, since the initial abundances are so different.



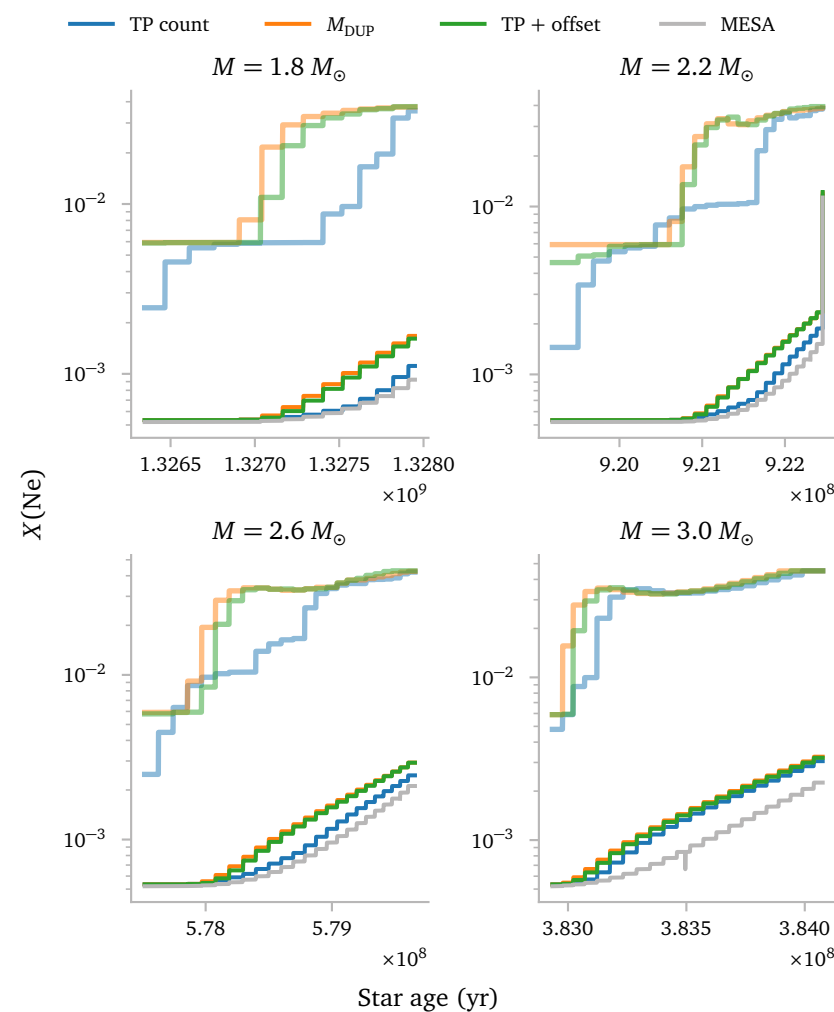
Compared to the Helium abundances, the MESA abundances agree much better with the Monash post-processing. The initial abundances are nearly identical. However, the Monash post-processing seems to predict lower enrichment.



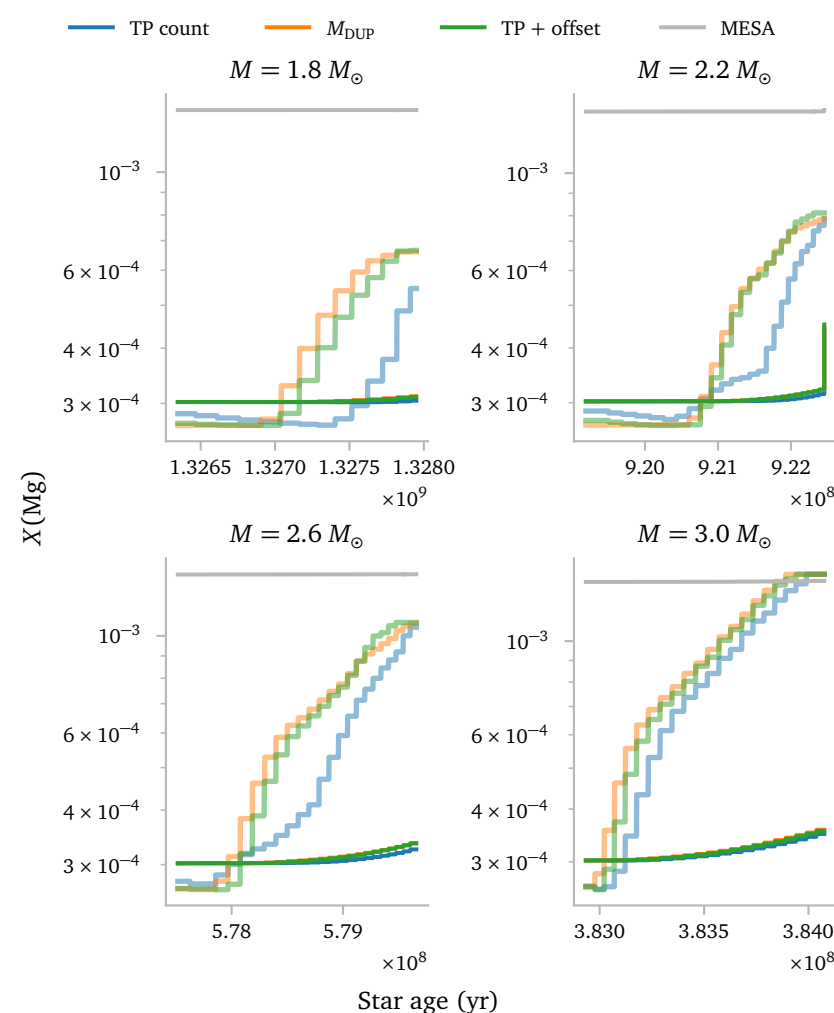
This one is uninteresting. Because the intershell abundance is much lower than the envelope abundance, the intershell values will slightly decrease due to dredge-up, but overall, not much happens.



The oxygen abundances are quite different. Initially, the oxygen abundances are slightly higher for the Monash post-processing abundances, but they grow much slower than the MESA ones. This is surprising. The MESA envelope abundance even reaches values close to the Monash intershell abundance values, which seems to imply that the MESA intershell abundance of oxygen is much larger than it is for the Monash models.

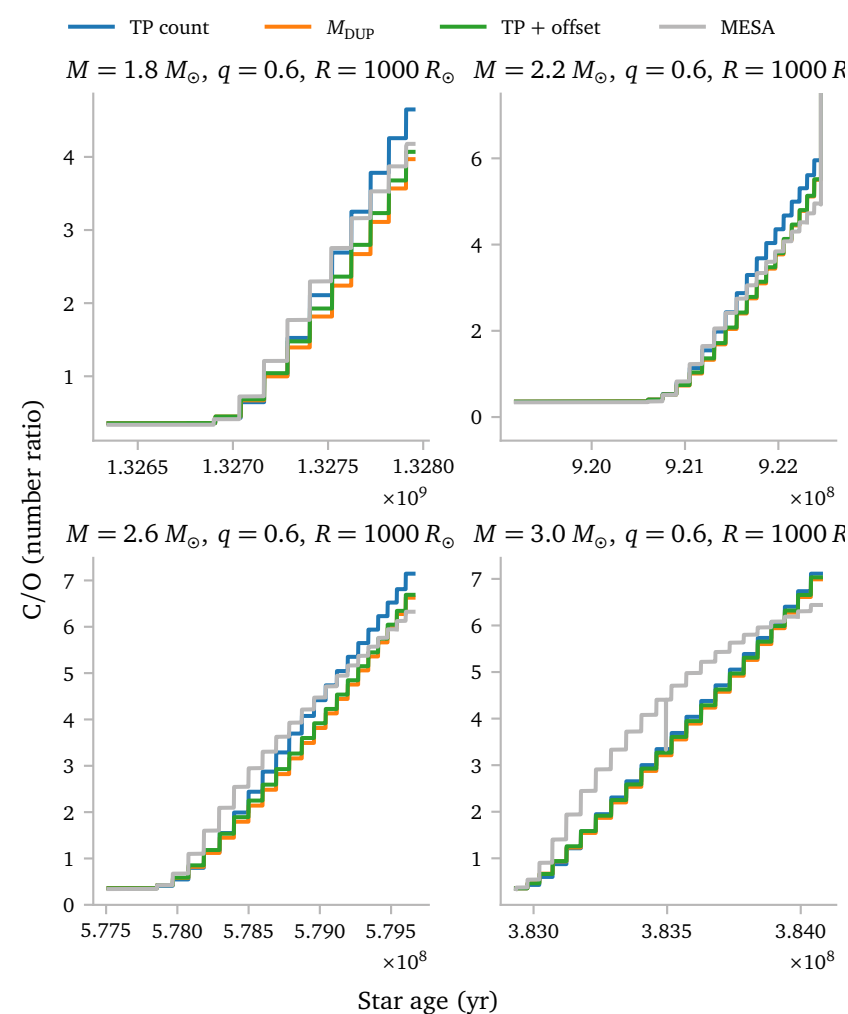


The Neon abundances are generally slightly larger for the Monash post-processed abundances than they are for the *MESA* models. In this case, the naive thermal pulse method actually produces the abundances that most closely match the *MESA* abundances, but this does not necessarily mean that this is the best method, since I am not sure how accurate the *MESA* abundances are.



The magnesium abundances are very different. Like I have already said, I think this is very much due to the limited reaction network.

Now that I have shown all abundances that are present in the *MESA* history data, we can also look at some ratios. The most interesting one is probably the C/O ratio:



Even though the absolute abundances for oxygen and carbon do not agree *super* well between the Monash abundances and the *MESA* abundances. However, the C/O ratio actually agrees very well, especially for  $M = 1.8 M_\odot$ ,  $M = 2.2 M_\odot$ , and  $M = 2.6 M_\odot$ .

e. Yields

Karakas & Lugaro (2016) define the yields of their Monash models as:

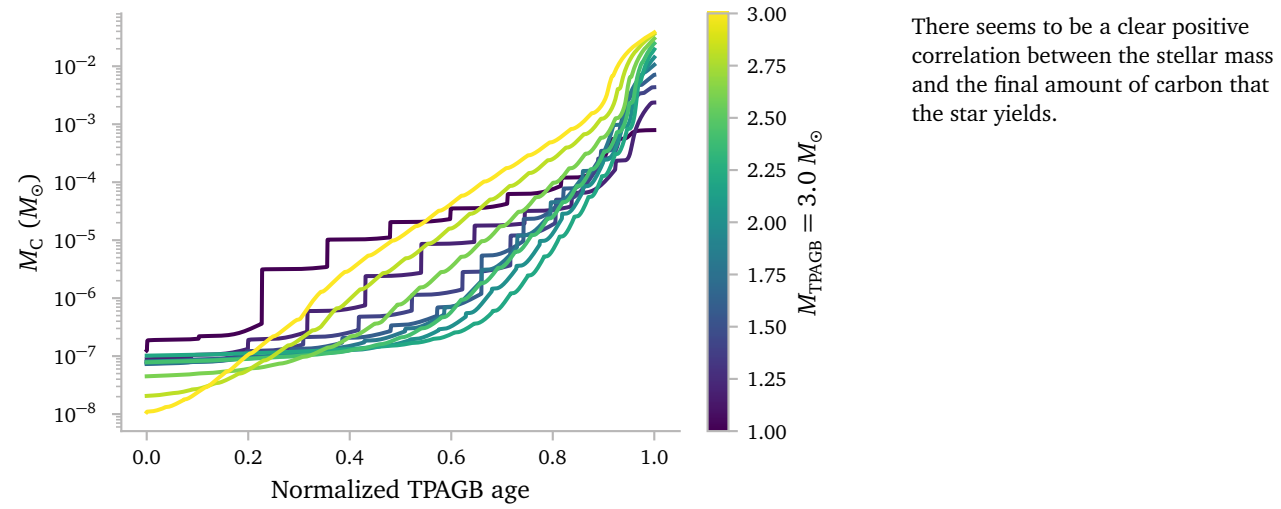
$$M_k = \int_0^\tau X(k) \frac{dM}{dt} dt,$$

where  $\tau$  is the lifetime of the stellar model. So, as far as I understand, this is simply the amount of mass of an element that is expelled.

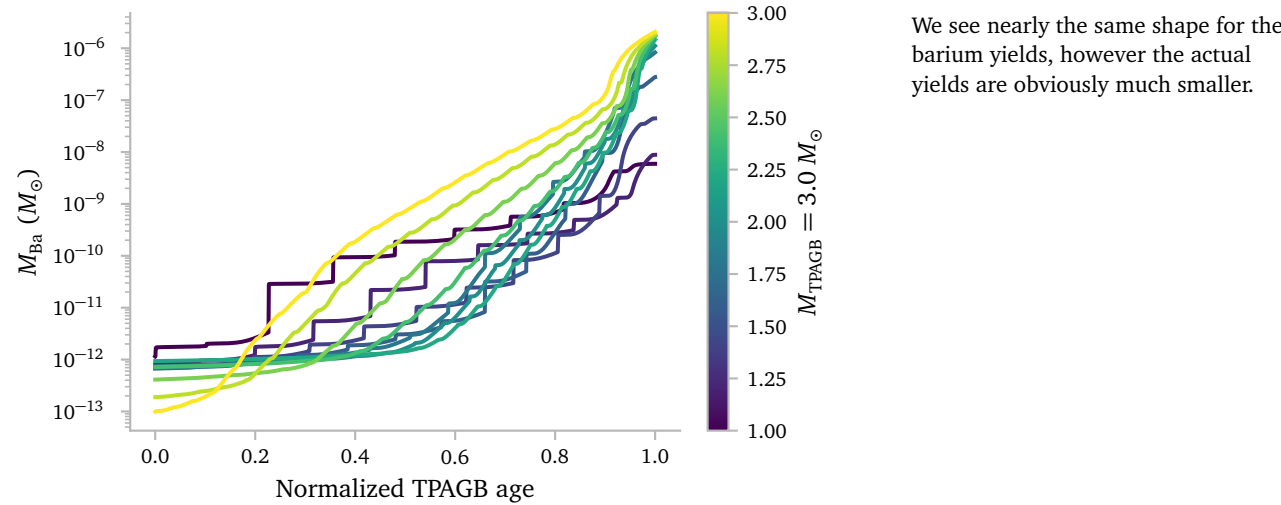
It is quite easy to compute the yields with my current setup, and Karakas & Lugaro (2016) published their yields so we can quite easily compare the two.

**NOTE:** I actually modified my Abundance class to also take in *just* the single star evolution from mesa. In this case, we cannot predict the actual accreted amount of mass. The accreted amount of mass is obviously not necessary to compute the yields.

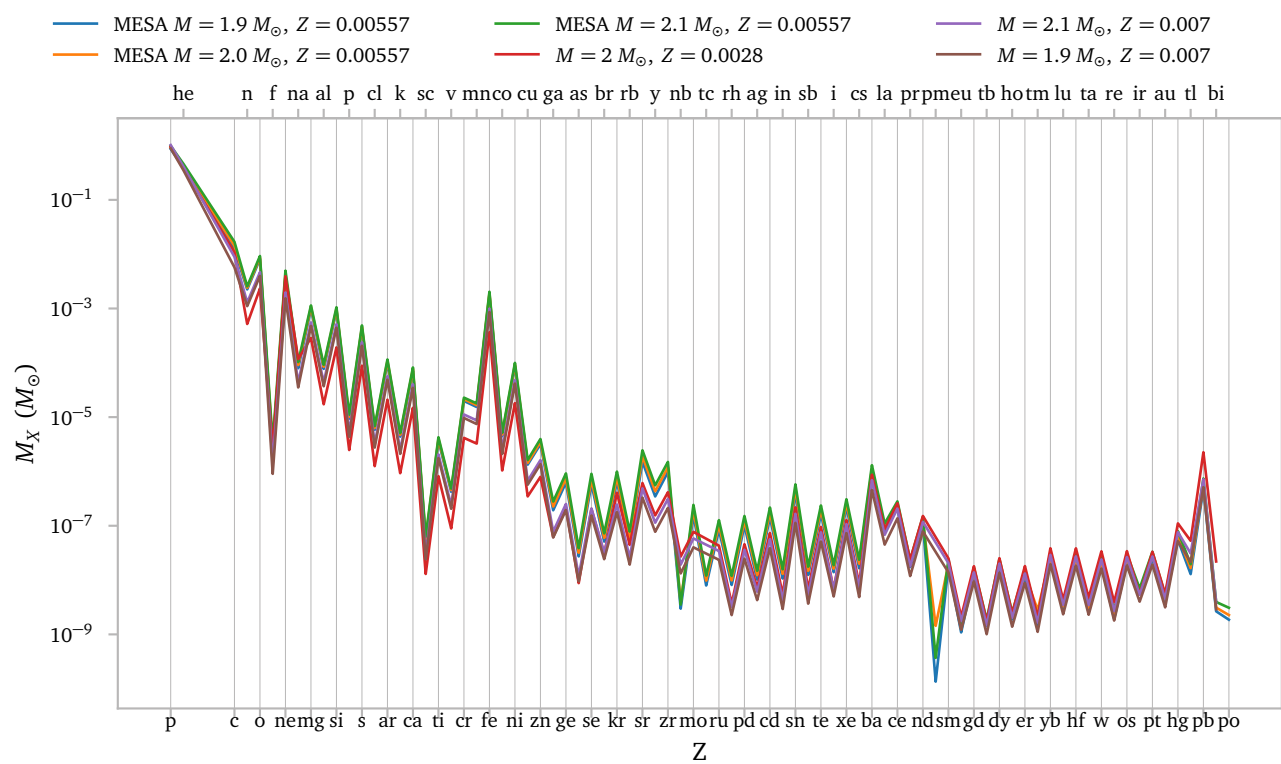
Below I plot the yields for carbon.



Below I plot the same plot, but now for barium:

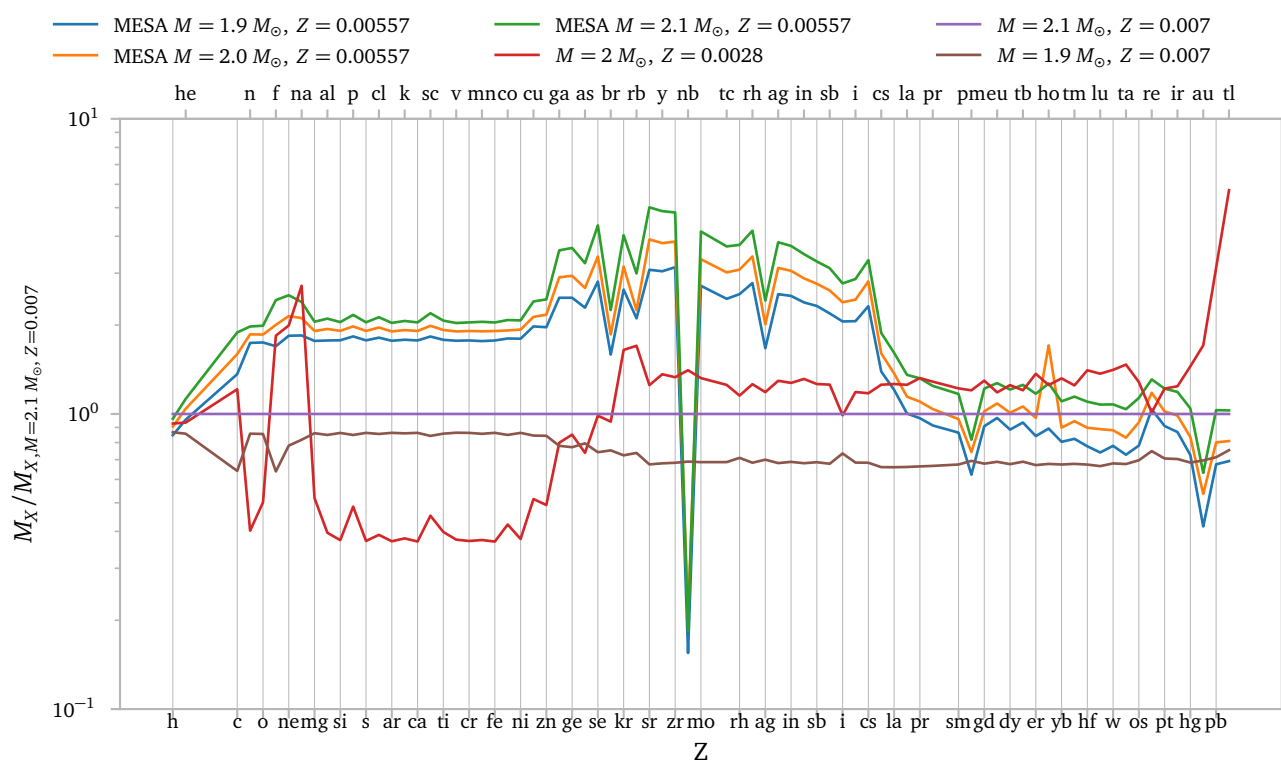


Below I plot the final yields for the MESA models and for the Monash models.



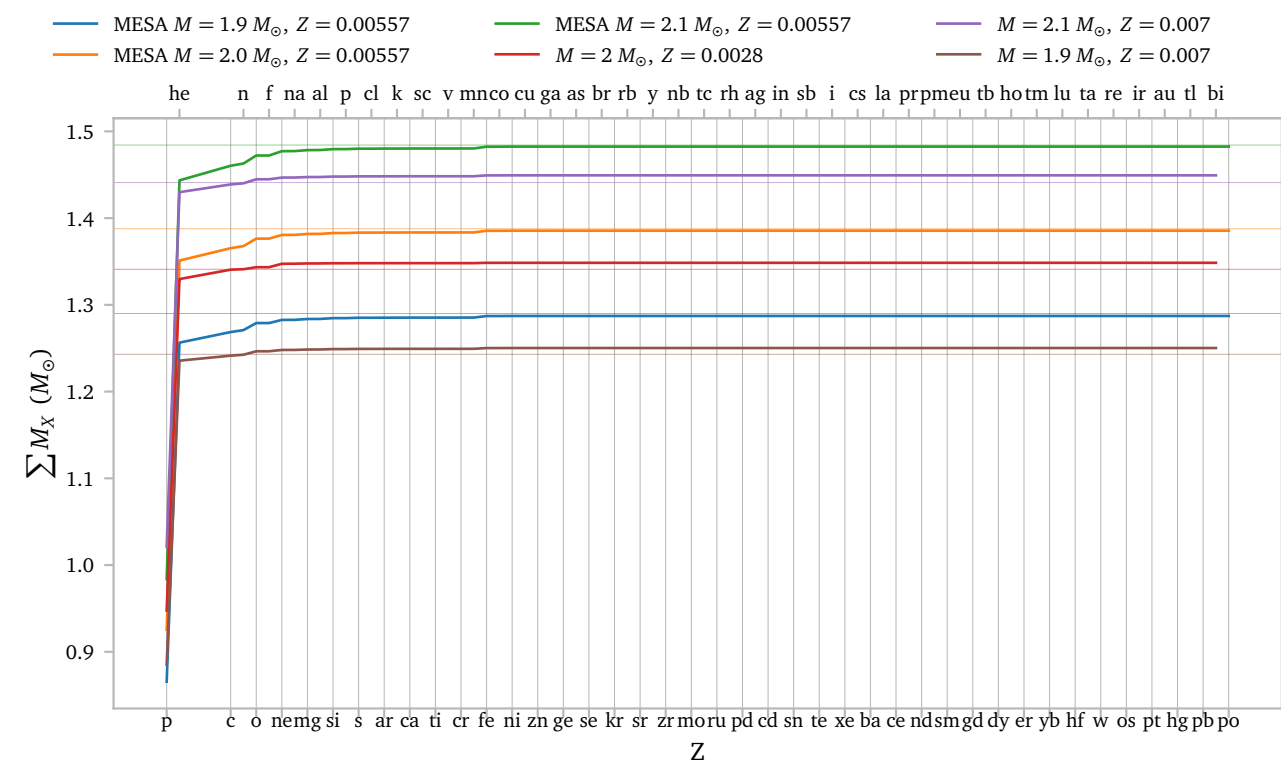
It looks like the MESA models always predict higher yields for elements lower than about neodymium. After that, the models agree very well. The monash yields also leave out Technetium, Promethium and Polonium. These all have no stable isotopes. It makes sense to skip these.

Below I divided the abundances by a chosen reference curve. In this case the purple line.

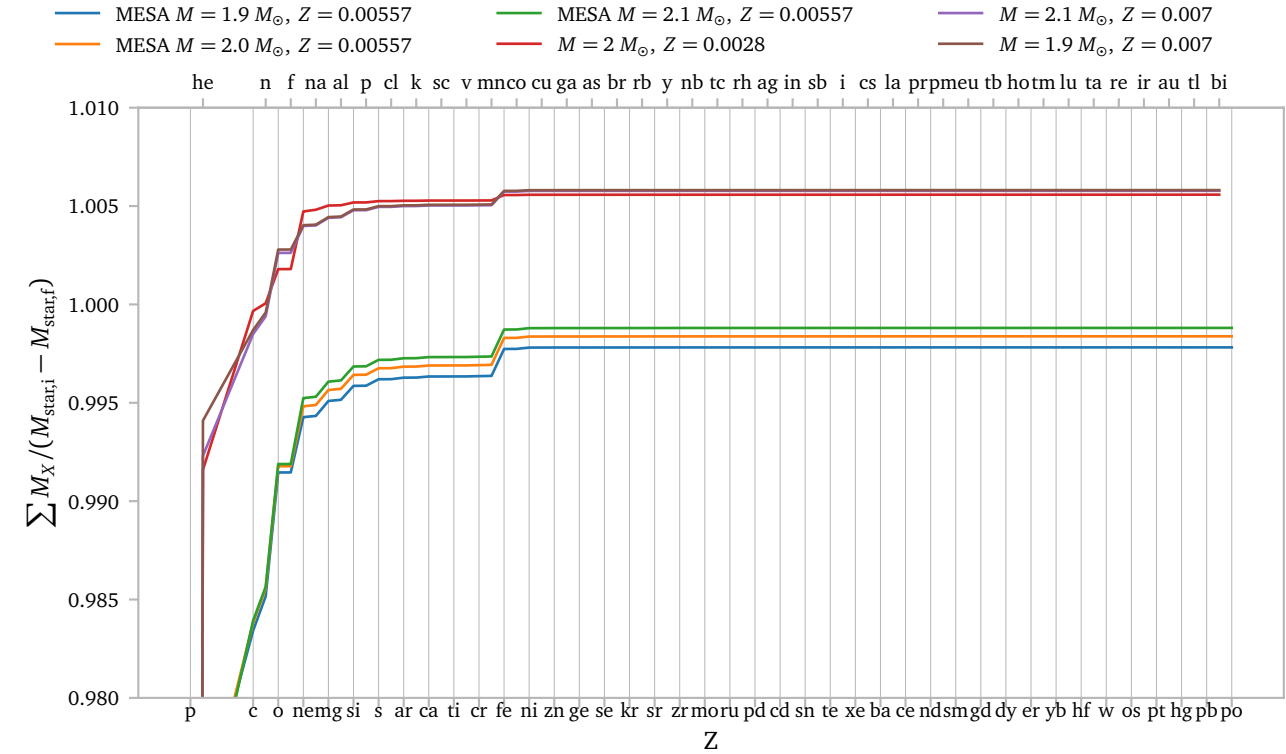


It looks like I am about a factor 2 off in the most cases, which seems quite concerning?

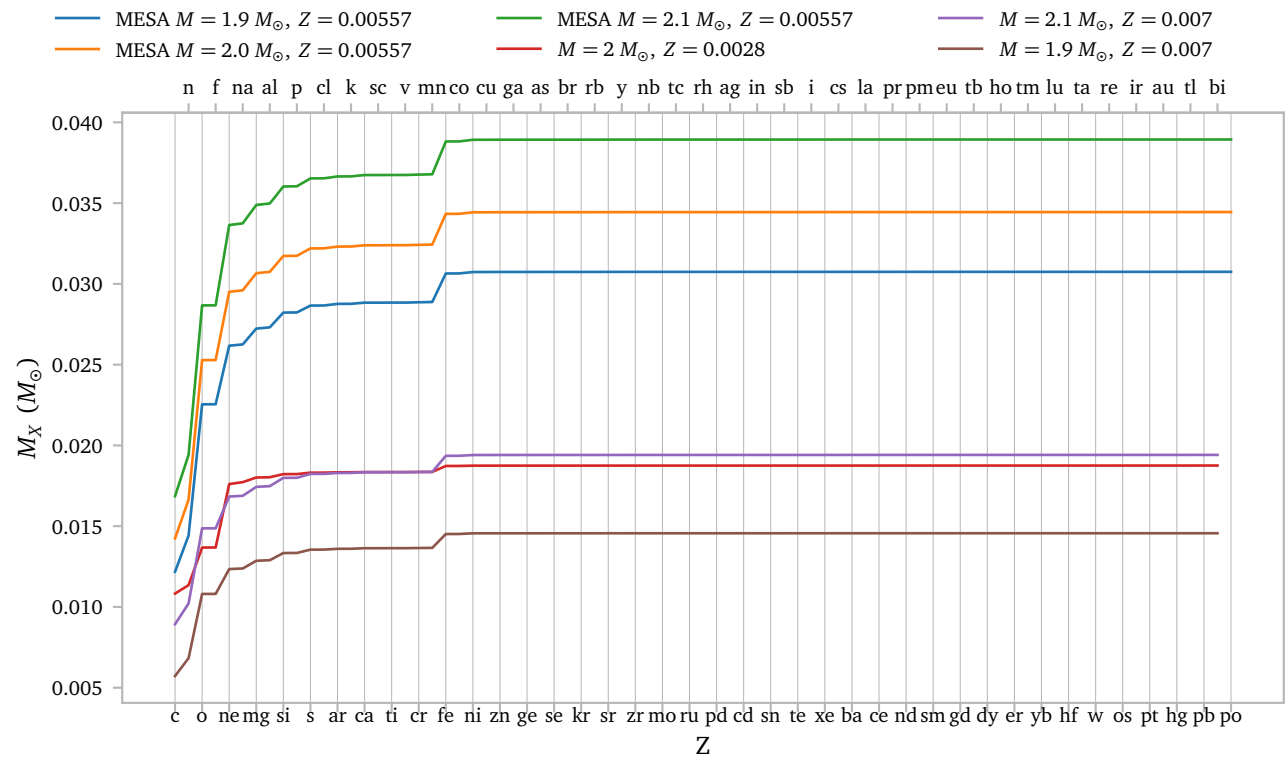
Because it looks like the MESA models nearly always predict values that are too large, I wonder if the total expelled mass is correct. Let me check this:



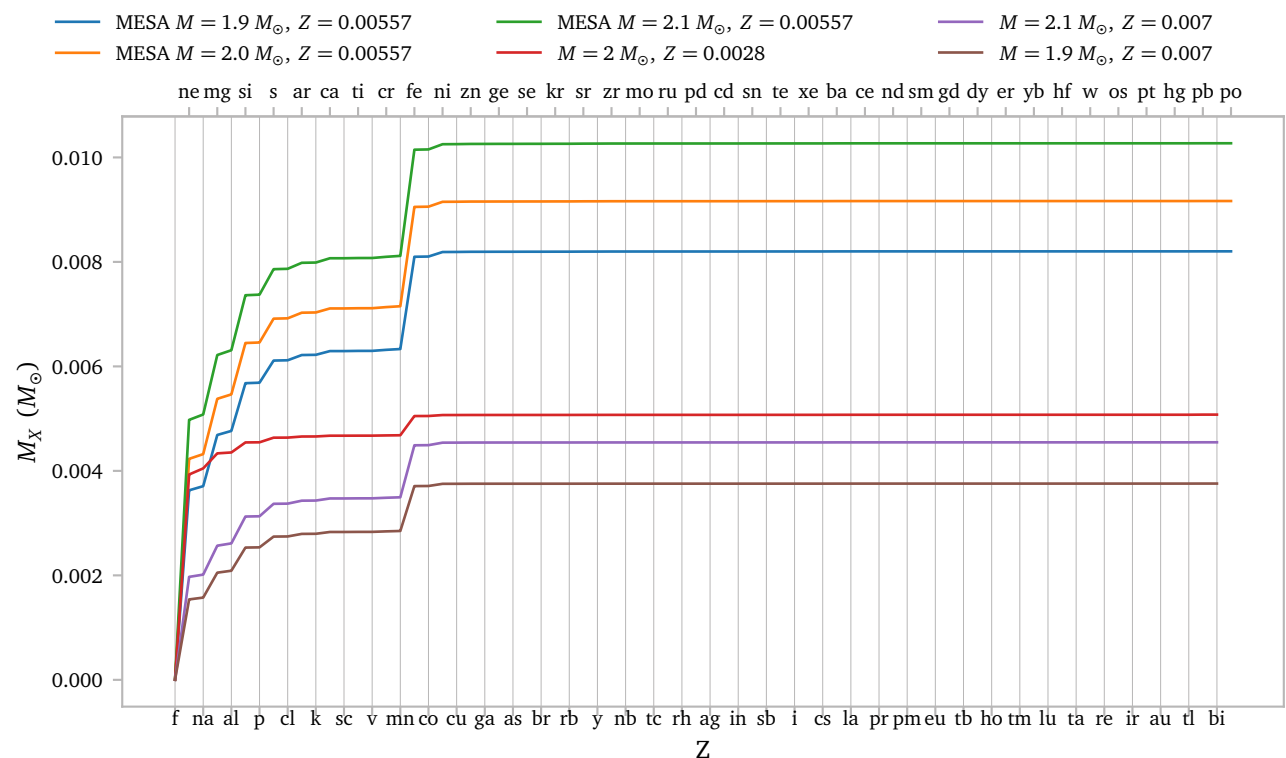
The MESA models seem to agree very well. The Monash models on average, overestimate the actual expelled mass.



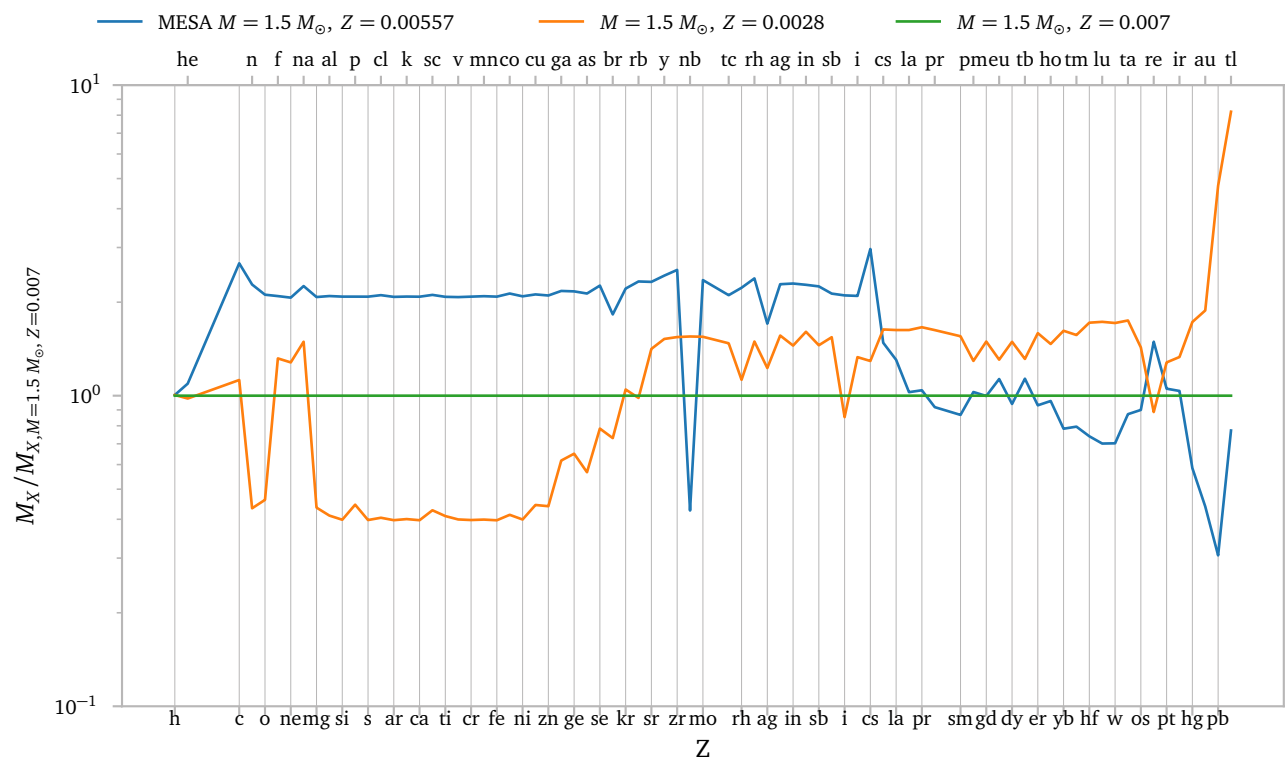
Below I plot the cumulative abundances for elements carbon and higher.



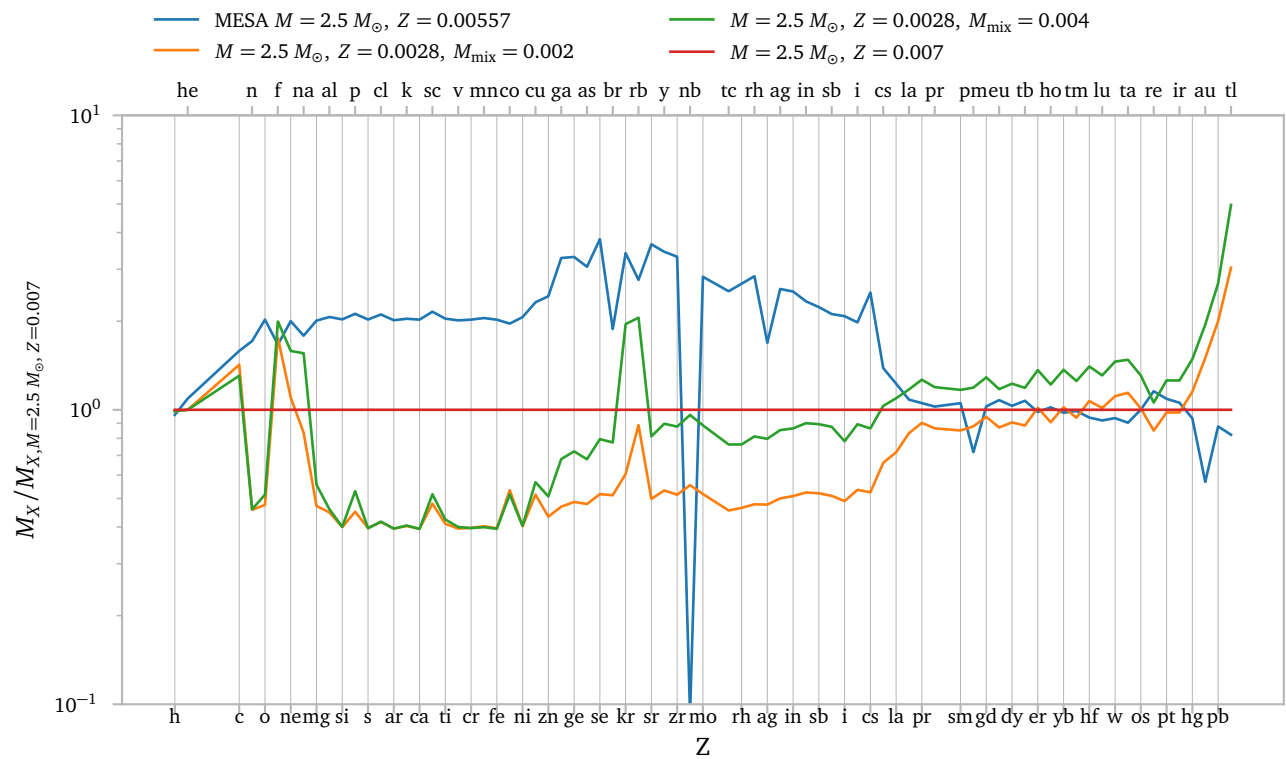
Below I plot the cumulative abundances for elements fluorine and higher.



Obviously, we can look at other masses too, but I think the pattern holds. Here I plot the expelled masses divided by a reference curve again:



Here I plot  $M = 2.5 M_{\odot}$ .



We see the same pattern for all models.

I think this all makes sense if we take into account that the *MESA* models have a significantly lower final core mass and significantly more dredge up, so they shed more of their envelope, which will generally be more enriched.

## References

Cinquegrana, G. C., Joyce, M., & Karakas, A. I. (2022) *Bridging the Gap between Intermediate and Massive Stars. I. Validation of MESA against the State-of-the-Art Monash Stellar Evolution Program for a  $2M_{\odot}$  AGB Star*. [ApJ939, 50](#).

Karakas, A. I. & Lugaro, M. (2016) *Stellar Yields from Metal-rich Asymptotic Giant Branch Models*. [ApJ825, 26](#).

Rees, N. R., Izzard, R. G., & Karakas, A. I. (2024) *Thermal pulses with MESA: resolving the third dredge-up*. [MNRAS527, 9643](#).

## A Optimizations

Last week, my code to compute the intershell abundances was very unoptimized. It worked but very often computations were repeated when it was not necessary. Now I have changed this, which makes the intershell enrichment computations quite quick. I think the speed-up is about a factor 10.

The main speed-up comes from caching the abundances per thermal pulse /  $M_{\text{DUP}}$  for every model into a binary file. Reading this file is *much* quicker than querying through the hole intershell abundances dataframe.

Below I give the new code.

```
class MonashModel:
    def __init__(self, M, Z, pulses, m_dup,
        ↳ intershell_isos):
        self.M = M
        self.Z = Z
        self.pulses = pulses
        self.m_dup = m_dup
        self.intershell_isos = intershell_isos

    index = np.where(self.m_dup > -4.5)[0][0]
    self.pulses_offset = self.pulses -
        ↳ self.pulses[index]

class Isotope:
    def __init__(self, isotope):
        self.key = isotope

        match isotope:
            case "n":
                self.mass = 1
                self.name = f"${\\texttrm{{{isotope}}}}$"
                self.short_name = "neutron"
            case "p":
                self.mass = 1
                self.name =
                    ↳ f"${\\texttrm{{{isotope}}}^{{{*}}}}$"
                self.short_name = "proton"
            case "d":
                self.mass = 2
                self.name = f"${\\texttrm{{{isotope}}}}$"
                self.short_name = "deuterium"
            case _:
                match = re.match(r"([a-zA-Z]+)(\\d+)$",
                    ↳ isotope)
                if match:
                    self.short_name = match.group(1)
                    self.mass = int(match.group(2))
                    self.name = (
                        ↳ f"${\\texttrm{{{self.short_name.capitalize()}}}_{{{self.mass}}}}$"
                    )
                else:
                    self.short_name = None
                    self.name = None
                    self.mass = None

    def __str__(self):
        return f"{self.key} with mass {self.mass}"

    def __repr__(self):
        return f"{self.key}"

class Element:
    def __init__(self, name):
        self.key = name
        self.name = name.capitalize()
        self.isotopes = {}

    def add_isotope(self, isotope):
        self.isotopes[isotope.key] = isotope

    def __getitem__(self, isotope):
        return self.isotopes[isotope]

    def __iter__(self):
        return iter(self.isotopes.values())

    def __repr__(self):
        return self.key

    def __getattr__(self, name):
        if name in self.isotopes:
            return self.isotopes[name]

class AbundanceTables:
    def __init__(self):

        with open("data/intershell_pd_df.pkl", "rb") as
            ↳ f:
            self.intershell = pickle.load(f)

        with open("data/env_pd_df.pkl", "rb") as f:
            self.envelope = pickle.load(f)

        with open("data/tp_info_pd_df.pkl", "rb") as f:
            self.tp = pickle.load(f)

        species = self.intershell.columns[4:]

        self.isotopes = {spec: Isotope(spec) for spec in
            ↳ species}
        self.elements = {}

        for isotope in self.isotopes.values():
            if isotope.short_name is None:
                continue

            element = self.elements.setdefault(
                isotope.short_name,
                ↳ Element(isotope.short_name)
            )
            element.add_isotope(isotope)

        self.envelope =
            ↳ self.envelope[self.envelope["pmz"] == 2e-3]
        self.envelope =
            ↳ self.envelope[self.envelope["N_ov"] != 0]
        self.envelope =
            ↳ self.envelope[self.envelope["Z"].astype(np.float64)
            ↳ == 0.014]
        self.envelope =
            ↳ self.envelope[self.envelope["M_init"].astype(np.float64)
            ↳ == 2]

    def __getattr__(self, name):
        try:
            return self.isotopes[name]
        except KeyError:
            raise AttributeError(f"{type(self).__name__}
                ↳ has no attribute {name!r}")

    def get_initial_envelope_abundance(self, element,
        ↳ metallicity):
        df = self.envelope[self.envelope["element"] ==
            ↳ element]
        return float(df["massfrac"].iloc[0]) *
            ↳ metallicity / 0.014

    def get_pulse_info_specific_model(self, mass,
        ↳ metallicity):
        df = self.tp[self.tp["initial_mass"] == mass]
        df = df[df["z"] == metallicity]
        return df
```

```
class Abundances:
    """
    this class computes and contains the abundances of
    ~ the envelope of a
    binary run. it uses the MESA binary simulation,
    ~ combined with the
    simple binary to determine
    """

    def __init__(self, model, df, method="m_dup"):
        self.model = model
        df_mix = df.intershell[df.intershell["pmz"] ==
                               ~ "2e-3"]
        self.df = df

        self.method = method

        simple =
        ~ get_star(full_path=self.model.params["single_star"])

        self.dup_simple = compute_m_DUP(simple)
        self.dup_detailed = compute_m_DUP(model,
        ~ self.dup_simple)

        binary_start_age = model.age[0]

        simple_age = np.asarray(simple.age)

        self.simple_end_idx = (
            np.searchsorted(
                simple_age,
                binary_start_age,
                side="right",
            )
            - 1
        )

        self.total_length = self.simple_end_idx +
        ~ len(self.model.age)
        self.m_dup = np.zeros(self.total_length)
        self.tp_count = np.zeros(self.total_length)
        self.m_env = np.concatenate(
            [simple.m_env[: self.simple_end_idx],
            ~ self.model.envelope_mass]
        )
        self.time = np.concatenate([simple.age[:
        ~ self.simple_end_idx], self.model.age])

        m2 = np.concatenate(
            [
                self.model.sb.m2[: self.simple_end_idx],
                self.model.star_2_mass,
            ]
        )

        valid = ~np.isnan(m2)

        m2_filled = np.nan_to_num(m2, nan=0)

        dm = np.diff(m2_filled)
        dm[~valid[:-1] | ~valid[1:]] = 0

        self.dm_acc = np.concatenate([[0], np.clip(dm, 0,
        ~ np.inf)])

        m1 = np.concatenate(
            [
                -1 * simple.mass[: self.simple_end_idx],
                -1 * self.model.star_1_mass,
            ]
        )

        valid = ~np.isnan(m1)

        m1_filled = np.nan_to_num(m1, nan=0)

        dm = np.diff(m1_filled)
        dm[~valid[:-1] | ~valid[1:]] = 0

        self.dm = np.concatenate([[0], np.clip(dm, 0,
        ~ np.inf)])

        for key, value in self.dup_simple.items():
            if value["index"] > self.simple_end_idx:
                break
            self.m_dup[value["index"]] = value["mass"]

        for key, value in self.dup_detailed.items():
            self.m_dup[self.simple_end_idx +
            ~ value["index"]] = value["mass"]

        self.tp_count[: self.simple_end_idx] =
        ~ simple.TP_count[: self.simple_end_idx]

        self.tp_count[self.simple_end_idx :] = (
            self.model.TP_count +
            ~ simple.TP_count[self.simple_end_idx]
        )

        self.monash_models = defaultdict(list)
        self._get_monash_masses_per_metallicity()
        self._prepare_monash_models()

    def __getattr__(self, name):
        if name in self.df.elements:

            # compute the intershell elemental abundance
            intershell = self.compute_intershell(name)

            # compute the initial envelope abundance
            envelope =
            ~ self.compute_envelope_abundance(name,
            ~ intershell)

            self.df.elements[name].envelope = envelope
            self.df.elements[name].intershell =
            ~ intershell
            self.df.elements[name].m_accreted =
            ~ np.cumsum(envelope * self.dm_acc)
            self.df.elements[name].m_yield =
            ~ np.cumsum(envelope * self.dm)

            return self.df.elements[name]

        if name in self.df.isotopes:

            return self.df.isotopes[name]

    def _prepare_single_monash_model(self, M, Z,
        ~ fresh=False):

        if not fresh:
            try:
                with open(
                    ~ f"/home/koen/master-internship/data/intershell-cache/M{M:.3f}Z{Z:.4f}.pkl",
                    "rb",
                ) as f:
                    monash_model = pickle.load(f)
                    return monash_model
            except FileNotFoundError:
                return
            ~ self._prepare_single_monash_model(M,
            ~ Z, True)

        else:
            intershell = self.df.intershell.query(
                f"Z == {Z} and pmz == 2e-3 and last == 1
                ~ and M1tp == {M}"
            ).sort_values("ntp")

            tp_info = self.df.tp.query(f"initial_mass ==
            ~ {M} and z == {Z}")

            intershell =
            ~ intershell[intershell.ntp.isin(tp_info.pulse)]

            # one intershell abundance per pulse
            i_data =
            ~ intershell.drop_duplicates("ntp").set_index("ntp")

            # one dredge-up mass per pulse
            tp_data =
            ~ tp_info.drop_duplicates("pulse").set_index("pulse")

            # common pulses, in ascending order
            pulses =
            ~ i_data.index.intersection(tp_data.index).sort_values()

            Mdredge = tp_data.loc[pulses,
            ~ "Ddredge"].to_numpy()
            isotope_abundance = np.log10(i_data.iloc[:,
            ~ 7:].clip(lower=1e-99))

            Mdredge = np.log10(np.cumsum(Mdredge) +
            ~ 1e-12)

            monash_model = MonashModel(M, Z, pulses,
            ~ Mdredge, isotope_abundance)

            with open(
                ~ f"/home/koen/master-internship/data/intershell-cache/M{M:.3f}Z{Z:.4f}.pkl",
                "wb",
            ) as f:
                pickle.dump(monash_model, f,
                ~ protocol=pickle.HIGHEST_PROTOCOL)

            return monash_model

    def _get_monash_masses_per_metallicity(self):
        try:
            with open(
                ~ f"/home/koen/master-internship/data/intershell-cache/MZ.pkl",
                "rb",
            ) as f:
                self._mass_Z_dict = pickle.load(f)
        except FileNotFoundError:
            self._mass_Z_dict = {}
            for Z in np.unique(self.df.intershell["Z"]):
                i = self.df.intershell.query(
                    f"Z == {Z} and pmz == 2e-3 and last
                    ~ == 1"
                ).sort_values("ntp")
                self._mass_Z_dict[Z] =
                ~ np.unique(i["M1tp"])
```

```
def _prepare_monash_models(self, Z=None):
    """
    this method combines the Monash Isotope dataset
    ~ and tp-info dataset and saves it as a
    ~ collection of simple dicts.
    it caches the result on disk using pickle.

    the dicts contain:
    M:      (float)
    Z:      (float)
    TPs:    numpy 1D array(floats)
    M_dup:  numpy 1D array(floats)
    isos:   328 x numpy 1D array(floats)

    """
    if Z == None:
        Z = self.model.params["z"]
    M = self.model.params["m"]

    if Z in [0.0028, 0.007, 0.014]:
        masses = self._mass_Z_dict[Z]

        x = M
        i = np.searchsorted(masses, x)

        if i == 0:
            lower = None
            upper = masses[0]
        elif i == len(masses):
            lower = masses[-1]
            upper = None
        else:
            lower = masses[i - 1]
            upper = masses[i]

        if lower != None:
            self.monash_models[Z].append(
                ~ self._prepare_single_monash_model(lower,
                ~ Z)
            )
        if upper != None:
            self.monash_models[Z].append(
                ~ self._prepare_single_monash_model(upper,
                ~ Z)
            )

    elif Z < 0.007:
        self._prepare_monash_models(Z=0.007)
        self._prepare_monash_models(Z=0.0028)
    else:
        self._prepare_monash_models(Z=0.014)
        self._prepare_monash_models(Z=0.007)

    # self._prepare_single_monash_model(M, Z)

def compute_intershell(self, name):
    # TODO: this needs to be changed to the ACTUAL
    ~ abundances
    # keep in mind that we need to sum the abundances
    ~ for the
    # different isotopes separately.
    # it is also important to keep in mind that the
    ~ abundances
    # here should be MASS RATIOS.

    intershell = np.zeros(self.total_length)

    match self.method:
        case "m_dup":
            interp_x = np.log10(np.cumsum(self.m_dup)
            ~ + 1e-12)

        case "tp":
            interp_x = self.tp_count

        case "tp offset":
            index = np.where(self.m_dup >
            ~ 10.0**-4.5)[0][0]
            interp_x = self.tp_count -
            ~ self.tp_count[index]
            print(interp_x)

    for isotope in self.df.elements[name].isotopes:

        intershell +=
        ~ self.df.elements[name].isotopes[
            isotope
        ].mass * self.get_abundance(
            self.df.isotopes[isotope],
            self.model.params["m"],
            self.model.params["z"],
            interp_x,
        )
    return intershell

def get_abundance(self, isotope, M, Z, interp,
    ~ drop=None):

    if Z in [0.0028, 0.007, 0.014]:
        return 10 ** self.get_abundance_Z(isotope, M,
        ~ Z, interp, drop)

    if Z <= 0.0028:
        return 10 ** self.get_abundance_Z(isotope, M,
        ~ 0.0028, interp, drop)
    if Z >= 0.014:
        return 10 ** self.get_abundance_Z(isotope, M,
        ~ 0.014, interp, drop)

    if Z <= 0.007:
        z_min = 0.0028
        z_max = 0.007
    else:
        z_min = 0.007
        z_max = 0.014

    abundance_min = self.get_abundance_Z(isotope, M,
    ~ z_min, interp, drop)
    abundance_max = self.get_abundance_Z(isotope, M,
    ~ z_max, interp, drop)
    weight = (np.log10(Z) - np.log10(z_min)) /
    ~ (np.log10(z_max) - np.log10(z_min))

    return 10 ** (abundance_min + weight *
    ~ (abundance_max - abundance_min))

def get_abundance_Z(self, isotope, M, Z, interp_x,
    ~ drop=None):

    match self.method:
        case "m_dup":
            attr = "m_dup"
        case "tp":
            attr = "pulses"
        case "tp offset":
            attr = "pulses_offset"

    if len(self.monash_models[Z]) == 1:
        abundance =
        ~ self.monash_models[Z][0].intershell_isos[isotope.key]
        x = getattr(self.monash_models[Z][0], attr)
        return np.interp(interp_x, x, abundance)

    abundance_min =
    ~ self.monash_models[Z][0].intershell_isos[isotope.key]
    x_min = getattr(self.monash_models[Z][0], attr)
    abundance_min = np.interp(interp_x, x_min,
    ~ abundance_min)
    mass_min = self.monash_models[Z][0].M
    abundance_max =
    ~ self.monash_models[Z][1].intershell_isos[isotope.key]
    x_max = getattr(self.monash_models[Z][1], attr)
    abundance_max = np.interp(interp_x, x_max,
    ~ abundance_max)
    mass_max = self.monash_models[Z][1].M

    weight = (M - mass_min) / (mass_max - mass_min)

    return abundance_min + weight * (abundance_max -
    ~ abundance_min)

def compute_envelope_abundance(self, name,
    ~ intershell):
    # INFO: gets the initial envelope abundance of
    ~ the element
    # scaled by the metallicity of the model.
    initial_envelope_abundance =
    ~ self.df.get_initial_envelope_abundance(
        element=name,
        metallicity=self.Z,
    )
    print(initial_envelope_abundance)

    # INFO: computes the elemental abundance in the
    ~ envelope by
    # enriching it with intershell abundances.
    envelope = np.zeros(self.total_length)
    print(intershell)
    delta_M_element = intershell * self.m_dup
    for i in range(self.total_length):
        if i == 0:
            envelope[i] = initial_envelope_abundance
            continue

        # envelope[i] = (envelope[i - 1] *
        ~ self.m_env[i] + delta_M_element[i]) / (
        # self.m_env[i] + self.m_dup[i]
        # )

        envelope[i] = (
            envelope[i - 1] * (self.m_env[i] -
            ~ self.m_dup[i]) + delta_M_element[i]
        ) / self.m_env[i]

    return envelope
```